

PG SOFT

对接文档

转账模式 v2.2.20



目录

目录	2
更新记录	4
1 概述	11
2 系统要求	12
2.1 游戏尺寸要求	12
2.2 硬件	12
2.3 浏览器	13
2.4 不支持的浏览器	13
3 游戏启动	14
3.1 游戏授权方式	14
3.1.1 HTML Scheme 方式	15
3.1.2 Android SDK Scheme 方式	30
3.1.3 iOS SDK Scheme 模式	43
3.1.4 API 参考	62
3.1.5 其他	64
3.2 请求示例	65
3.2.1 直接启动 HTML 游戏模式	65
3.2.2 iframe 模式 (推荐)	66
3.2.3 Ajax 模式 (不推荐)	69
3.2.4 POSTMAN (RAW 主体)	71
3.2.5 CURL command	72
3.2.6 故障排除方式	73
3.3 游戏启动限制	76
3.4 常见问题	77
4 启动游戏 JS 库	78
4.1 避免浏览器弹出窗口受阻挡	78
4.2 实现方式	79

5	对接 API	81
5.1	运营商提供对接 API.....	81
5.1.1	API 格式.....	81
5.1.2	令牌验证	86
5.2	PG SOFT 提供对接 API.....	89
5.2.1	API 格式.....	89
5.2.2	查询玩家的钱包余额	91
5.2.3	充值与转出流程	99
5.2.4	钱包操作限制及处理	106
5.2.5	转出所有余额（任选项）	107
5.2.6	获取单个交易记录（任选项）	110
5.2.7	投注详情页面（任选项）	112
5.2.8	获取多个玩家钱包余额（任选项）	116
5.2.9	其他（任选项）	119
5.3	PG SOFT 游戏对接运营商 iOS 应用程序.....	120
6	解决待处理投注.....	121
7	重置游戏状态.....	123
8	限制.....	124
8.1	IP 限制.....	124
8.2	货币限制.....	125
9	附录.....	126
9.1	错误代码.....	126
9.2	货币代码.....	128
9.3	货币基础单位及处理	134
9.4	投注类型.....	136
9.5	交易类型.....	137
9.6	平台.....	138
9.7	语言.....	139

更新记录

版本	更新日期	描述
2.2.0	2019-11-09	修订文件初版
2.2.1	2020-03-09	5.1.1 令牌验证 • 添加了关于 operator_player_session 参数的描述 5.2.10 获取单个交易记录 • 修订 API 返回参数
2.2.2	2020-09-24	3.2 游戏登陆限制 • 添加了新内容 5.2.4 钱包操作限制及处理 • 添加了新内容
2.2.3	2020-10-02	5.2.3 充值与转出流程 • 添加了新错误代码 5.2.3.1 充值 • 添加了新错误代码 5.2.3.2 转出余额 • 添加了新错误代码 5.2.11 获取单个交易记录 • 添加了新错误代码
2.2.4	2021-03-17	在 API URL 添加了跟踪 Id, 并修订 API URL 例子 • 5.2.1 登录游戏 • 5.2.2 查询钱包余额 • 5.2.3.1 充值 • 5.2.3.2 转出余额 • 5.2.5 创建玩家账号 (任选项) • 5.2.6 踢出玩家 (任选项) • 5.2.7 冻结玩家 (任选项) • 5.2.8 恢复玩家账号 (任选项)

		5.2.9 查看玩家状态（任选项） 5.2.10 转出所有余额（任选项） 5.2.11 获取单个交易记录（任选项） 5.2.3.12 投注详情页面（任选项）
2.2.5	2021-03-18	5.2.10 转出所有余额（任选项） - 移除在 API 请求中的 amount 参数 - 修订 API 请求参数例子 - 修订 API 返回参数例子
2.2.6	2021-03-22	9.2 货币代码 修订货币名称
2.2.7	2021-03-28	添加新 API 5.2.13 获取用户钱包
2.2.8	2021-05-20	5.2.2 查询钱包余额 - 添加了新 API 返回参数 - 修订 API 返回参数例子 8.2 货币限制 - 修订货币限制列表 9.6 游戏 - 修订游戏列表 5.2.10 转出所有余额 - 修订 API 注解
2.2.9	2021-10-13	添加新内容 5.1.1.3 哈希校验 修订限制的国家 8.1 IP 限制 修订 API URL 和 API URL 示例中跟踪 ID 5.1.2 令牌验证 添加了新错误代码 9.1 错误代码 修订浏览器支持版本

		2.3 浏览器 修订请求 3.1.1 URL Scheme 方式 移除列表 9.6 游戏
2.2.10	2022-06-27	修订描述 3.1.1 URL Scheme 方式 5.1.1.3.1 标头字段 添加新货币 9.2 货币 添加新内容 5.2.4 钱包操作限制及处理 2.4 不支持的浏览器 移除累积奖池内容 5.2.14 其他（任选项） 修订描述以及 URL 例子 5.1.1.3.1 标头字段 3.1.2 Web 大厅方式 移除 API 请求参数的 ct 参数 3.1.1 URL Scheme 方式 添加新语言 9.6 语言 修订 API URL 5.2.11 投注详情页面（任选项） 移除参数类型 3.1.2 Web 大厅方式 移除玩家相关 API 5.2.6 创建玩家账号（任选项） 5.2.7 踢出玩家（任选项） 5.2.8 冻结玩家（任选项） 5.2.9 恢复玩家账号（任选项） 5.2.10 查看玩家状态（任选项）
2.2.11	2022-07-22	修订参数名称 3.1.2 Web 大厅方式

		4.2 实现方式 修订内容及移除 PHP 货币 8.2 货币 移除 DEC 及 TNT 货币 9.2 货币代码 修订内容 3.1.1.1 现实查核 移除 rurl 参数 3.1.1 URL Scheme 方式 修订内容 5.1.1.3.1 标头字段 添加新内容 5.2.7 获取单个交易记录（任选项）
2.2.12	2023-05-31	添加新内容及新请求参数 3.1.1 URL Scheme 方式 3.1.2 Web 大厅方式 5.1.2 令牌验证 移除游戏授权方式 3.1 WebView 方式 3.1.3 WebView 方式（仅适用于 PG SOFT 应用程序和 PG SOFT 大厅） 5.2.2 游戏登录 移除 ESP、HRK、STD、TRL 及 AUD 货币 9.2 货币
2.2.13	2023-09-26	移除以下内容 3.1 游戏授权方式 3.1.1 URL Scheme 方式 3.1.2 Web 大厅方式 添加新内容及请求参数 3.1 游戏授权方式 3.1.1 HTML Scheme 方式 3.1.1.1 实现方式 3.1.1.2 GetLaunchURLHTML API

		修订描述 3.1 游戏授权方式 3.1.1.3 现实查核 4.1 避免浏览器弹出窗口受阻挡 4.2 实现方式 5.1.2 令牌验证 5.1.1.3 哈希校验
2.2.14	2024-02-29	添加新内容 3.1.2 Android SDK Scheme 方式 3.1.3 iOS SDK Scheme 方式 3.1.4 API 参考 3.1.5 其他 9.2 货币 修订标题及描述 3.1.1.2 GetLaunchURLHTML API 3.1.1.3 现实查核 3.2 请求示例 3.3 游戏登录限制 5.1.3.1 获取用户钱包 5.2.3 充值与转出流程 移除描述 5.2.4 钱包操作限制及处理
2.2.15	2024-03-20	修订标题及描述 3.1.2.1 实现方式 3.1.3.1 实现方式 4.1 避免浏览器弹出窗口受阻挡 4.2 实现方式 5.2.7 投注详情页面 (任选项) 删除描述 9.3 投注类型
2.2.16	2024-04-02	修订标题及描述 3.1.2.1 实现方式 3.1.3.1 实现方式

		8.1 IP 限制
2.2.17	2025-02-18	添加新内容及请求参数 3.1.1.2 GetLaunchURLHTML API 5.1.2 令牌验证 5.2.3.1 充值 5.2.3.2 转出余额 5.2.5 转出所有余额（任选项） 5.2.7 投注详情页面（任选项） 5.2.8 获取多个玩家钱包余额（任选项） 修订描述 2.2 硬件 5.1.1.3 哈希校验（任选项） 5.2.2 查询玩家的钱包余额 5.2.3 充值与转出流程 8.2 货币限制 9.2 货币代码 9.3 货币基础单位和处理 移除 JPY 和 GBP 货币 9.2 货币代码 移除 iwk 参数 3.1.1.2 GetLaunchURLHTML API 添加新内容 3.1.1.3 故障排除 3.2 请求示例 3.4 常见问题 移除香港添加英国 8.1 IP 限制 添加新限制货币和区域 8.1 IP 限制 9.2 货币代码

2.2.18	2025-09-11	删除描述 3.1.1.2 GetLaunchURLHTML API 修订描述和内容 3.1.2.2 GetLaunchURLSDK API 5.1.1.3 哈希校验 (任选项) 5.2.2 查询玩家的钱包余额 5.2.3 充值与转出流程 5.2.5 转出所有余额 (任选项) 5.2.6 获取单个交易记录 (任选项) 5.2.7 投注详情页面 (任选项) 5.2.8 获取多个玩家钱包余额 (任选项) 8.2 货币限制 9.2 货币代码 添加新货币代码和语言 9.2 货币代码 9.7 语言
2.2.19	2026-03-27	修订描述 5.2.6 获取单个交易记录 (任选项) 移除 BGN 和 CSD 货币 9.2 货币代码 修订内容 2.2 硬件 8.2 货币代码 添加新内容 2.3 浏览器
2.2.20	2026-06-18	移除 player_name 参数 5.2.6 获取单个交易记录 (任选项)

1 概述

本文档详细介绍转账模式的对接方式。所谓转账模式，即指玩家需要从玩家钱包将金额转账 PG SOFT 游戏系统钱包才可进行游戏。这也表示，玩家需将盈利的金额从 PG SOFT 游戏系统钱包转账至中心钱包后才可进行提款。

2 系统要求

2.1 游戏尺寸要求

PG SOFT 游戏以 H5 制作并以垂直方式呈现，在尺寸规格上有最低要求。有关 iframe 中的游戏尺寸，请参阅以下内容：

- 框架理想尺寸：1280 x 720
- 最小尺寸：540 x 960
- 最大尺寸：1080 x 2340

2.2 硬件

	Android	iOS	Windows	macOS
操作系统版本	5.0 或以上	11.0 或以上	7 或以上	10.13 或以上
内存	3GB	3GB	4GB	4GB
中央处理器	Snapdragon 710 或以上	iPhone X 或以上	• 1 千兆赫 (GHz) 或更快 • 基于 32-bit (x86) 或 64-bit (x64) 处理器 • 双核处理器或以上	Intel 双核处理器 (兼容图形) 或 Apple 芯片
储存	4GB 或更大	4GB 或更大	20GB 或更大	20GB 或更大

2.3 浏览器

平台	浏览器	支持版本	备注
桌面	Microsoft Edge	79 或以上	基于 Chromium 的 Microsoft Edge
	Firefox	58 或以上	仅限桌面端
	Google Chrome	59 或以上	
	Safari	11.1 或以上	
	其他基于 Chromium 的浏览器	Chromium 59 等效版本或以上	不包括 UC 浏览器、百度浏览器、夸克浏览器、CM 浏览器
Android	Google Chrome	59 或以上	
	Microsoft Edge	Chromium 59 等效版本或以上	
	其他基于 Chromium 的浏览器	Chrome 59 等效版本或以上	不包括 CM 浏览器
iOS	Safari	11.1 等效版本或以上	原生 WebKit 浏览器
	Google Chrome / Microsoft Edge / Opera	Safari 11.1 等效版本或以上	iOS 浏览器均使用 WebKit 内核，而非 Chromium

2.4 不支持的浏览器

- Android : 火狐浏览器及 CM 浏览器
- iOS : CM 浏览器
- 桌面 : UC 浏览器、百度浏览器、夸克浏览器、CM 浏览器、IE 浏览器

3 游戏启动

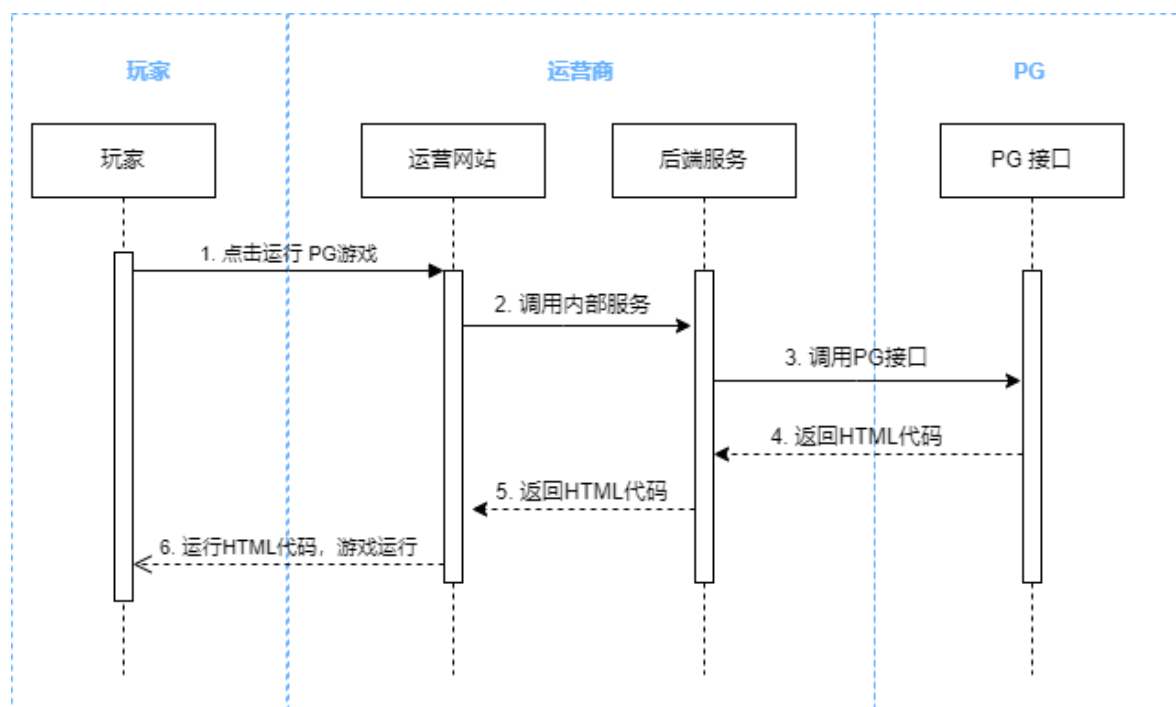
3.1 游戏授权方式

PG SOFT 支持 1 种登录授权方式：

- **HTML Scheme 方式**适用于 iOS、Android 和浏览器，它使用令牌参数进行授权和登录。

3.1.1 HTML Scheme 方式

HTML Scheme 方式是游戏启动的常规方法。玩家需要通过运营商的网站登录，并通过游戏大厅页面启动游戏。



1. 玩家访问运营商网站，并点击 PG SOFT 游戏。
2. 运营商前端调用后端服务，以取得 HTML 代码。
3. 后端服务收到前端请求后，调用 PG SOFT GetLaunchURLHTML API。
4. PG SOFT API 返回 HTML 代码。
5. 后端将 HTML 代码返回前端。
6. 前端接收 HTML 代码后，运行该 HTML 代码，从而在玩家的浏览器上显示 HTML 内容（运行 PG SOFT 游戏）。

***注：请将步骤 4 中 PG SOFT 返回的 HTTP 标头和响应主体直接传递到步骤 5 中的玩家浏览器，切勿进行任何处理，以避免发生未知错误。**

优点:

PG SOFT 将会根据请求参数（如 client_ip、group_id 等）返回一个或多个游戏启动 URL，然后通过玩家侧检测选择最佳的游戏启动 URL 进行游戏。

若遇到域名问题，PG SOFT 也可单方面切换游戏服务器地址，而无需通知运营商配合更换，从而减少运营商的负担。

为何淘汰 URL Scheme 方式？

1. 现今网络状况极为复杂，目前市场上没有任何一个服务供应商能够完美模拟不同玩家的网络状况。传统方案已经无法有效地解决玩家遇到的各种网络问题。
2. 为了确保每位玩家都能获得最佳的游戏体验，PG SOFT 推出了 HTML 方案。我们将从玩家的角度出发，对 API 返回的每个游戏启动 URL 进行测试，确保测试过程对运营商保持透明的同时，减轻运营商团队的技术负担。
3. 针对不同地区所发生的网络异常（如域名阻断、域名劫持等），PG SOFT 将根据实际情况对返回的 HTML 代码进行调整及优化。
4. 为了提升域名安全性并确保运营商的持续运营，PG SOFT 还实施了 HTML 加密，以防止第三方对域名进行阻断或劫持。

3.1.1.1 实现方式

步骤 1

运营商须通过自身后端来调用 PG SOFT GetLaunchURLHTML API，以获取 HTML 页面代码。

*** 注：**运营商的 API 响应中需包含以下标头，以防止响应被存储在浏览器缓存中：

Cache-Control: no-cache, no-store, must-revalidate

标头参数：

***以下说明有可能随着 API 升级而进行调整，仅供参考。**

HTTP 标头	备注
Content-Type: text/html	响应的 HTTP 主体格式
Content-Length	响应的 HTTP 主体长度
Transfer-Encoding: chunked	服务器也可能使用 Transfer-Encoding 来表示返回内容的长度。绝大多数客户端仍然支持这种格式。
Content-Encoding: gzip	表示响应的 HTTP 主体使用了 gzip 压缩
Cache-Control: no-cache, no-store, must-revalidate	浏览器缓存控制，用以防止浏览器对响应的 HTTP 主体进行缓存。

标头例子：

```
200 OK HTTP/1.1
Content-Type: text/html
Content-Encoding: gzip
Content-Length: 2910
Cache-Control: no-cache, no-store, must-revalidate
Server: nginx/1.24.0
Date: Thu, 16 Nov 2023 07:43:39 GMT
```

步骤 2

获取 HTML 页面代码后，通过网络浏览运行该 HTML 代码。

请参考以下运营商大厅例子：

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.4/jquery.min.js">
</script>
<script>
    $(document).ready(function(){
        $("button").click(function(){
            $.post( "https://<运营商后端服务>/api",
                { param1: "value1", param2: "value2" },
                function(data) {
                    var newPage = window.open('', '_blank');
                    newPage.document.open();
                    newPage.document.write(data); // 运行 HTML 代码
                    newPage.document.close();
                });
        });
    });
</script>
</head>
<body>
<button>运行游戏</button>
</body>
</html>
```

3.1.1.2 GetLaunchURLHTML API

运营商须通过自身后端来调用 PG SOFT GetLaunchURLHTML API，以获取 HTML 页面代码。

请求参数

API URL : {PgSoftAPIDomain}/external-game-

launcher/api/v1/GetLaunchURLHTML

HTTP Method : POST

URL 参数:

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符 (GUID) 注: 请把参数值设置为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
operator_token	varchar(20)	是	运营商独有的身份识别
path	varchar(500)	是	游戏路径: /<游戏 ID>/index.html Web 大厅路径: /web-lobby/{面板类型} 锦标赛路径: /tournament/ 投注详情页面路径: /history/redirect.html 注: - 请使用 UriEncode 对值进行编码, 以避免未知错误 - 游戏 ID 是每个游戏'的唯一标识 - 面板类型 首次加载时 Web 大厅的显示面板

¹ 请从 PG SOFT 获取最新的列表

			[NO VALUE] : 默认面板 (锦标赛) tournament/open : 锦标赛 – 开 放报名的面 板 tournament/process : 锦标赛 – 正 在进行的的面 板 tournament/end : 锦标赛 – 结 束的面板 games : 仅显示游戏
extra_args	Query String Parameter	是	游戏运行和投注详情页面参数 (请参考相关参数列表) 注: - 请使用 UrlEncode 对值进行编码, 以避免未知错误 - 最多 500 字符
url_type	varchar(50)	是	URL 类型: game-entry: 游戏入口 web-lobby : 游戏 Web 大厅入口 web-tournament: 锦标赛入口 web-history: 投注详情页面
client_ip	varchar(50)	是	玩家 IP
group_id	UInt16	否	运营商分组 ID

extra_args 参数（游戏启动）：

参数名称	参数类型	必需项	描述
btt	UInt8	是	游戏启动模式 ²
ops	varchar(200)	是	运营商系统生成的令牌 注： - 请使用 <code>UrlEncode</code> 对值进行编码，以避免未知错误 - 最多 200 字符 - 请将参数值设置为 GUID 格式或任何随机字符串
l	varchar(50)	否	游戏显示语言 ³ 默认：英文
te	UInt16	否	运营商运行健康提醒的时间，以秒为单位（现实查核 ⁴ ）
ri	UInt16	否	健康提醒的间隔时间，以秒为单位（现实查核 ⁴ ）
op	String	否	运营商的自定义参数，PGSoft API 将在验证运营商玩家会话时包含参数值。 注：请使用 <code>UrlEncode</code> 对值进行编码，以避免未知错误
f	String	否	游戏退出 URL 默认：重定向到 PG SOFT 退出页面 注： - 请使用 <code>UrlEncode</code> 对值进行编码，以避免未知错误 - 分配数值到 <code>PGGameCloseUrl</code> 以关闭游戏窗口

² 请参考[投注类型](#)的部分以获取完整列表

³ 请参考[语言](#)的部分以获取完整列表

⁴ 请参考[现实查核](#)的部分以获取更多信息

oc	Integer	否	启动游戏时检查屏幕方向 ⁵ 0: 跳过屏幕方向检查 1: 普通模式 (默认)
----	---------	---	---

⁵ 请参考截图

extra_args 参数 (Web 大厅) :

参数名称	参数类型	必需项	描述
ops	varchar(200)	是	运营商系统生成的玩家令牌 注: - 请使用 UrlEncode 对值进行编码, 以避免未知错误 - 最多 200 字符 - 请将参数值设置为 GUID 格式 或任何随机字符串
l	varchar(50)	否	Web 大厅的显示语言 ⁶ 默认: 英文
width	Integer	否	Web 大厅的页面宽度 默认值: 响应式 数值范围: 数值≥930

extra_args 参数 (锦标赛) :

参数名称	参数类型	必需项	描述
ops	varchar(200)	是	运营商系统生成的玩家令牌 注: - 请使用 UrlEncode 对值进行编码, 以避免未知错误 - 最多 200 字符 - 请将参数值设置为 GUID 格式 或任何随机字符串
l	varchar(50)	否	Web 大厅的显示语言 ⁶ 默认: 英文

⁶ 请参考[语言](#)的部分以获取完整列表

extra_args 参数（投注详情页面⁷）：

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符（GUID） 注：请把参数值设为 GUID 格式
t	String	是	运营商令牌
psid	String	是	母注单 ID
sid	String	是	投注 ID
gid	String	否	游戏 ID
lang	String	否	语言： • en: 英文（默认） • zh: 中文
type	String	是	固定值：operator

例子：

主体：

operator_token=xxxx&path=%2F1%2Findex.html&extra_args=btt%3D1%26ops%3D12345-abcd-1234-abcd-12345&url_type=game-entry&client_ip=1.1.1.1

⁷ 请参考[投注详情页面](#)的部分以获取完整信息

返回结果

HTTP 响应主体：

浏览器只需执行 HTML 页面即可启动游戏。

***注: 返回的 HTML 代码内容会根据实际情况进行调整及优化，并无固定格式。请勿通过反序列化或解析方式取得最终的游戏启动 URL。**

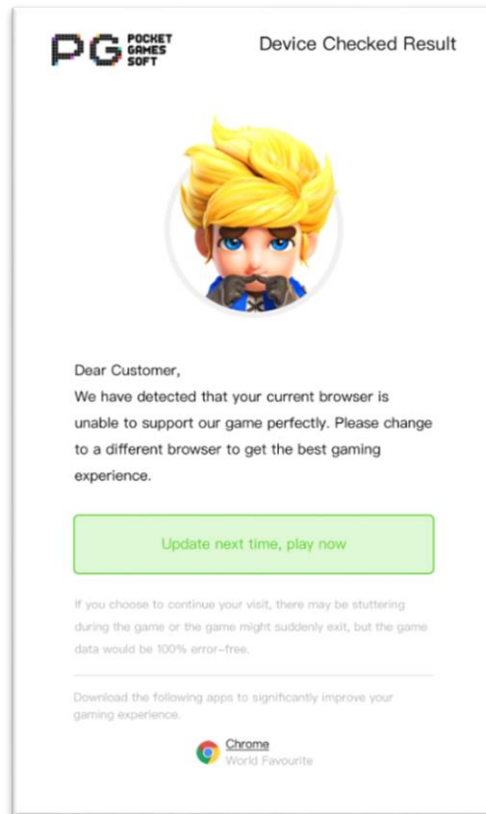
返回的 HTTP 状态码：

状态码	描述	原因	解决办法
200	成功返回	-	-
400	无效参数	参数未按照要求提供	检查 HTTP 方法、HTTP 标头和 HTTP 主体是否按照要求进行请求。
404	路径未找到	无效路径	检查路径是否正确
500	内部服务器错误	发生内部服务器错误	• 重试 • 通知 PG SOFT
504	网关超时	请求未能成功到达目标服务器，且某个代理服务器发生了故障	• 重试 • 通知 PG SOFT

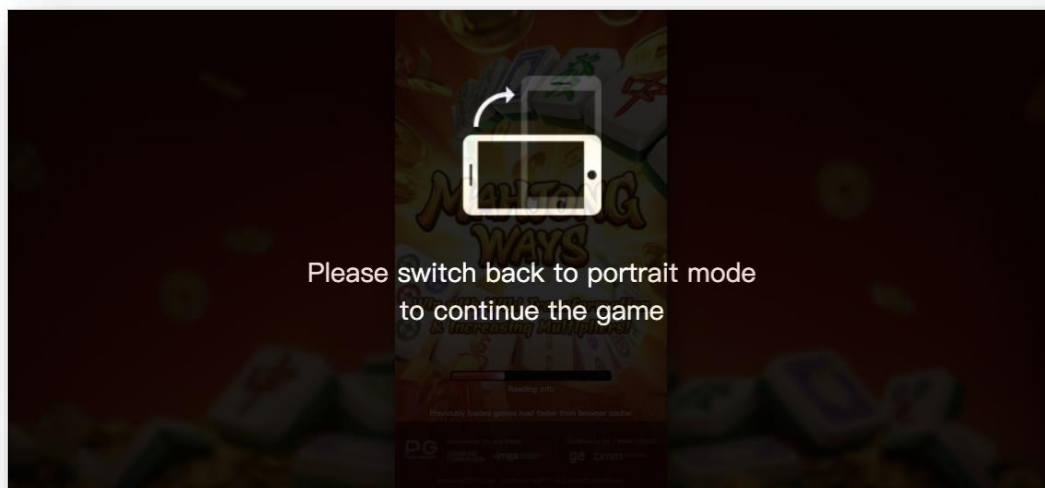
HTML 响应例子（仅供参考）：

```
<!doctype html><html lang="en">...</html>
```

截图



设备兼容性检查



屏幕方向检查

3.1.1.3 故障排除

以下情境可能导致玩家无法顺利进入游戏：

- 域名解析失败（例如：被劫持、被封锁或被其他干扰所致）
- PG SOFT 网络异常
- 玩家端网络连接失败
- 系统版本过旧（可能导致 SSL 证书验证失败）
- 浏览器限制 JavaScript 执行
- 浏览器设置禁止图片下载
- 浏览器限制页面跳转
- 浏览器安装了广告拦截插件
- 使用非主流浏览器或浏览器版本过旧

3.1.1.4 现实查核

于 HTML scheme⁸ 的方式中, 运营商可将 “te” 和 “ri” 的参数包含在启动游戏的 URL 以进行现实查核。

- te : 计算第一次现实查核所使用的时间（以秒为单位）
- ri : 现实查核的间隔时间（以秒为单位）

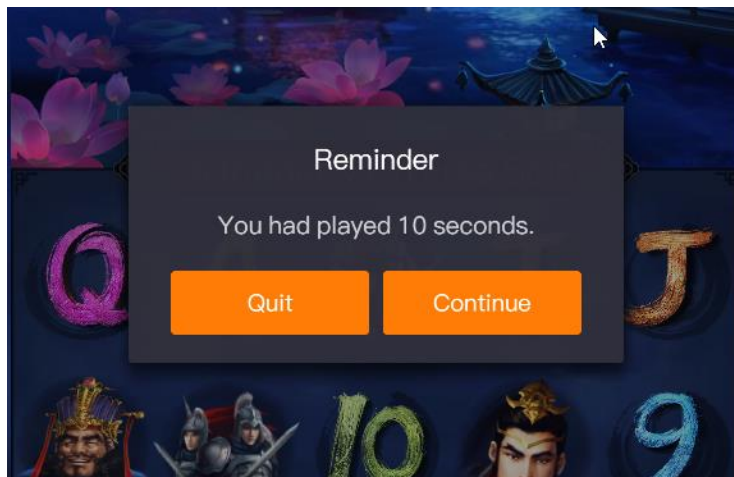
例子：

ri=300&te =290

第一个现实查核提醒将在 10 秒后显示（300s-290s = 10s），后续提醒将每 5 分钟显示一次。

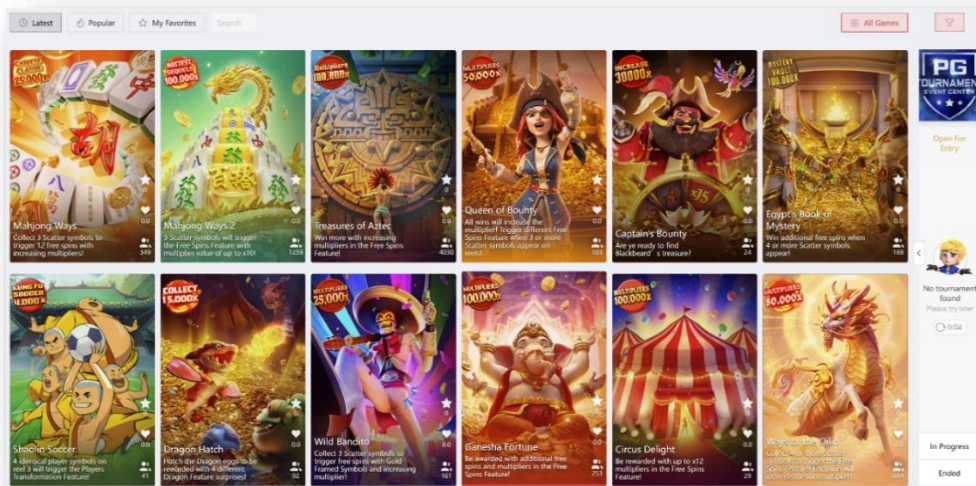
⁸ 请参考 [HTML Scheme 方式](#) 的部分以获取更多信息

截图



健康提醒

截图



Web 大厅方式

3.1.2 Android SDK Scheme 方式

3.1.2.1 实现方式

步骤 1: 预先配置

1. 从 PG SOFT 取得 SDK AAR 文件。
2. 调用 API⁹以获取 JSON 响应。

注:

测试 SDK 文件: <https://app.pg-staging.com/game-sdk-android/game-sdk.zip>

*注: 生产 SDK 文件请向 PG SOFT 索取。

*注: 返回的 HTML 代码内容会根据实际情况进行调整及优化, 并无固定格式。请勿通过反序列化或解析方式取得最终的游戏启动 URL。

Cache-Control: no-cache, no-store, must-revalidate

标头参数:

*以下说明有可能随着 API 升级而进行调整, 仅供参考。

HTTP 标头	备注
Content-Type: text/html	响应的 HTTP 主体格式
Content-Length	响应的 HTTP 主体长度
Transfer-Encoding: chunked	服务器也可能使用 Transfer-Encoding 来表示返回内容的长度。绝大多数客户端仍然支持这种格式。
Content-Encoding: gzip	表示响应的 HTTP 主体使用了 gzip 压缩
Cache-Control: no-cache, no-store, must-revalidate	浏览器缓存控制, 用以防止浏览器对响应的 HTTP 主体进行缓存。

⁹ 请参考 [GetLaunchURLSDK API](#) 以获取更多信息

标头例子:

```
200 OK HTTP/1.1
Content-Type: application/json
Content-Length: 2910
Cache-Control: no-cache, no-store, must-revalidate
Server: nginx/1.24.0
Date: Thu, 16 Nov 2023 07:43:39 GMT
```

步骤 2: 导入 SDK 和相关依赖项

1. 导入 [dnsjava](#) 依赖项。
2. 导入 SDK Android AAR 文件到项目中:

```
// build.gradle
dependencies {
    implementation files("libs/pg-gamesdk-release.aar")
    implementation group: 'dnsjava', name: 'dnsjava', version: '3.5.2'
    ...
}
```

3. (建议) 将 Android builder.gradle minSdk 设置为至少 **26**。

步骤 3: 使用 Android SDK

通过 GameWebViewFragmentActivity 启动游戏

SDK 提供了 GameWebViewFragmentActivity, 使用户能够快速启动游戏。

1. 调用 API¹⁰获取 JSON 响应, 并通过 GameWebViewConfigModel.Builder 构建 SDK 的 Webview 配置。
2. 使用 GameWebViewConfigModel.Builder 构建一个 SDK 的 DNS 配置。
3. 通过 intent.putExtra(<GameWebViewFragmentActivity.KEY>,Bundle)将 DNS 和 GameWebView 配置传递给 GameWebViewFragmentActivity, 以启动游戏:

¹⁰ 请参考 [GetLaunchURLSDK API](#) 的部分以获取更多信息

```
// Main Activity

// data return from API
String response = "{\"error\":null,\"data\":...}"
Intent intent = new Intent(MainActivity.this,
GameWebViewFragmentActivity.class);

Bundle gameWebViewConfigBundle = new GameWebViewConfigModel.Builder()
                                .sdkResponse(responseData)
                                .locale(GameSDKLocale.ZH)
                                .toBundle();
intent.putExtra(GameWebViewFragmentActivity.GAME_WEB_VIEW_CONFIG_BUNDLE,
gameWebViewConfigBundle);

Bundle dnsConfigBundle = DNSConfigInstance.GOOGLE_DNS_SERVER.getBundle();
intent.putExtra(GameWebViewFragmentActivity.DNS_CONFIG_BUNDLE,
dnsConfigBundle);

getApplicationContext().startActivity(intent);
```

自定义 GameWebViewFragmentActivity

若要在 FragmentActivity 中通过 FragmentManager 来加载 GameWebView，请参考以下代码：

***注：GameWebViewFragment 需要在 onCreate 生命周期之后的回调中进行提交。**


```
// CustomFragmentActivity / CustomAppCompatActivity
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.demo_fragment_activity);
}

@Override
protected void onStart() {
    GameWebViewFragment webViewFragment = new
    GameWebViewFragment(DNSConfigInstance.GOOGLE_DNS_SERVER, GameSDKLocale.ZH);

    getSupportFragmentManager().beginTransaction()
        .add(R.id.demo_fragment_container, webViewFragment,
    GameWebViewFragment.class.getName())
        .commitNow();

    // data return from API
    String response = "{\"error\":null,\"data\":...}"
    webViewFragment.loadGame(response);
}

// remove GameWebViewFramgment and clean up
private void removeView() {

    getSupportFragmentManager().beginTransaction().remove(webViewFragment).comm
    itNow();
    webViewFragment = null;
}
```

```
// demo_fragment_activity.xml
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/demo_fragment_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
</androidx.constraintlayout.widget.ConstraintLayout>
```

自定义 DNS

SDK 默认提供四个 DNS 服务器以解析域名：

```
DNSConfigInstance.GOOGLE_DNS_SERVER;  
DNSConfigInstance.CLOUDFLARE_DNS_SERVER;  
DNSConfigInstance.TENCENT_DNS_SERVER;  
DNSConfigInstance.ALI_DNS_SERVER;
```

若想使用其他自定义的 DNS 服务器可通过 `DNSConfigModel.Builder` 来创建 SDK 所需要的 DNS 配置。

希望了解更多关于 DNS 解析和 DoH (DNS over HTTPS) 的信息，请点击以下链接：

- [什么是 DNS](#)
- [CloudFlare 公共 DNS 地址](#)
- [使用 CloudFlare DNS over HTTPS](#)

IP 和 DoH 域名必须不同。只有 PG SOFT 内部产品的域名才会由 DoH 解析为 IP，而其他域名，如 `google.com`，则将由公共 DNS 进行解析。

```
DNSConfigModel customDnsModel = new DNSConfigModel.Builder()  
    .ip("1.1.1.1")  
    .domain("https://1.0.0.1/dns-query")  
    .headerMap(new HashMap<String, String>()  
    {{  
        put("Accept", "application/dns-json");  
    }}).build();  
GameWebViewFragment webViewFragment = new GameWebViewFragment(customDnsModel);  
...
```

SDK 异常回调

- GameWebViewFragment 提供 setOnErrorReceive 来获取 GameWebView 异常、VPN 配置异常、或是 GameSDKResponseModel 异常。
- GameSDKJsonResponseException 由 GameSDKResponseModel 抛异常，通常在响应数据内容不正确时才会抛出此异常。
- GameSDKSetupError 由 GameWebView 或 GameVPNService 抛异常，通常在 DNSConfigModel 或 GameWebViewConfigModel 传递无效或错误值时才会抛出此异常。
- GameSDKCommonError 由 GameWebViewFragment 或 GameVPNService 抛异常，通常在数据验证通过但包含不正确的数据（例如无效的 DNS 服务器或游戏 URL）时才会抛出此异常。

```
webViewFragment.setOnErrorReceive(new GameWebViewErrorNotify() {  
    @Override  
    public void onErrorReceive(Exception e) {  
        if (e instanceof GameSDKJsonResponseException) {  
            e.printStackTrace();  
        } else {  
            throw new RuntimeException(e);  
        }  
    }  
});
```

SDK 本地化

- 在 [Android 13](#) 之前，应用本地化和区域设置并没有一个统一标准。而 GameSDK 提供了 GameSDKLocale，让开发者可以自行设置 GameWebView 中使用的语言。
- SDK 提供了符合 ISO 639-1 标注的语言代码（如 en、zh），并使用 GameSDKLocale 进行打包。
- 请在[这里](#)查阅 GameSDKLocale 语言代码表。
- 请参考 GameSDKLocale 中提供的区域静态变量，以获取当前 SDK 支持的语言列表。

```
// if language code not support, it will return GameSDKLocale.EN by default.  
GameSDKLocale locale = GameSDKLocale.getGameSDKLocale("en");  
new GameWebViewConfigModel.Builder()  
    .sdkResponse(responseData)  
    .locale(locale)  
    .build();  
  
// or direct set while new GameWebViewFragment  
GameWebViewFragment webViewFragment = new  
GameWebViewFragment(DNSConfigInstance.GOOGLE_DNS_SERVER, GameSDKLocale.EN);
```

3.1.2.2 GetLaunchURLSDK API

运营商须通过自身后端调用此 PG SOFT GetLaunchSDK API 以获取 JSON 格式的加密重定向地址数据。

请求参数

API URL : {PgSoftAPIDomain}/external-game-launcher/api/v2/GetLaunchURLSDK
HTTP Method : POST

URL 参数:

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符 (GUID) 注: 请把参数值设为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
operator_token	varchar(20)	是	运营商独有的身份识别
path	varchar(500)	是	游戏路径: /<游戏 ID>/index.html Web 大厅路径: /web-lobby/{面板类型} 锦标赛路径: /tournament/ 投注详情页面路径: /history/redirect.html 注: - 请使用 UriEncode 对值进行编码, 以避免未知错误 - 游戏 ID 是每个游戏¹¹的唯一标识符 - 面板类型 首次加载时 Web 大厅的显示面板

¹¹ 请从 PG SOFT 获取最新的列表

			[NO VALUE] : 默认面板 (锦标赛) tournament/open : 锦标赛 – 开放报名的面板 tournament/process: 锦标赛 – 正在进行的的面板 tournament/end : 锦标赛 – 结束的面板 games : 仅显示游戏
extra_args	Query String Parameter	是	游戏运行参数 (请参考相关参数列表) 注: - 请使用 UrlEncode 对值进行编码, 以避免未知错误 - 最多 500 字符
url_type	varchar(50)	是	URL 类型: game-entry: 游戏入口 web-lobby : 游戏大厅入口 web-tournament: 锦标赛入口
client_ip	varchar(50)	是	玩家 IP
group_id	UInt16	否	运营商分组 ID

extra_args 参数 (游戏启动) :

参数名称	参数类型	必需项	描述
btt	UInt8	是	游戏启动模式 ¹²
ops	varchar(200)	是	运营商系统生成的令牌 注: - 请使用 UrlEncode 对值进行编码, 以避免未知错误 - 最多 200 字符

¹² 请参考[投注类型](#)的部分以获取完整列表

			请将参数值设置为 GUID 格式或任何随机字符串
l	varchar(50)	否	游戏显示语言 ¹³ 默认：英文
te	UInt16	否	运营商运行健康提醒的时间，以秒为单位（现实查核 ¹⁴ ）
ri	UInt16	否	健康提醒的间隔时间，以秒为单位（现实查核 ¹⁴ ）
op	varchar(200)	否	运营商的自定义参数，PG SOFT API 将在验证运营商玩家会话时包含参数值。 注：请使用 UriEncode 对值进行编码，以避免未知错误
f	varchar(200)	否	游戏退出 URL 默认：重定向到 PG SOFT 退出页面 注： - 请使用 UriEncode 对值进行编码，以避免未知错误 - 分配数值到 <i>PGGameCloseUrl</i> 以关闭游戏窗口
oc	UInt8	否	启动游戏时检查屏幕方向 0: 跳过屏幕方向检查 1: 普通模式（默认）

extra_args 参数（Web 大厅）：

参数名称	参数类型	必需项	描述
ops	varchar(200)	是	运营商系统生成的令牌 注： - 请使用 UriEncode 对值进行编码，以避免未知错误 - 最多 200 字符

¹³ 请参考[语言](#)的部分以获取完整列表

¹⁴ 请参考[现实查核](#)的部分以获取更多信息

			请将参数值设置为 GUID 格式或任何随机字符串
l	varchar(50)	否	Web 大厅显示语言 ¹⁵ 默认: 英文
width	Integer	否	Web 大厅的页面宽度 默认: 响应式 值范围: 值 ≥ 930

extra_args 参数 (锦标赛) :

参数名称	参数类型	必需项	描述
ops	varchar(200)	是	运营商系统生成的令牌 注: - 请使用 <code>UrlEncode</code> 对值进行编码, 以避免未知错误 - 最多 200 字符 - 请将参数值设置为 GUID 格式或任何随机字符串
l	varchar(50)	否	Web 大厅显示语言 ¹⁶ 默认: 英文

extra_args 参数 (投注详情页面¹⁷) :

参数名称	参数类型	必需项	描述
t	String	是	运营商令牌
psid	String	是	母注单 ID
sid	String	是	投注 ID
gid	String	否	游戏 ID
lang	String	否	语言: - en: 英文 (默认) - zh: 中文

¹⁵ 请参考[语言](#)的部分以获取完整列表

¹⁶ 请参考[语言](#)的部分以获取完整列表

¹⁷ 请参考[投注详情页面](#) 的部分以获取完整信息

type	String	是	固定值: operator
------	--------	---	---------------

例子:

主体:

operator_token=xxxx&path=%2F1%2Findex.html&extra_args=btt%3D1%26ops%3D12345-abcd-1234-abcd-12345&url_type=game-entry&client_ip=1.1.1.1

返回结果

HTTP 响应主体:

响应将是 JSON 数据。

***注: JSON 响应格式有可能随时更新, 故仅供参考之用。**

成功返回:

参数名称	参数类型	描述
error	object	发生错误时返回的错误消息; 若未发生错误, 值将为 "null" 。
data	string	混淆后的输出内容
version	string	当前版本为 2, 后续可能会随着版本升级而变更。

失败返回:

参数名称	参数类型	描述
code	int	错误码。错误码与 HTTP 状态码一致。 400: 请求参数错误 500: 内部服务器错误
message	string	错误的详情
tid	string	Trace ID 是请求的唯一标识符 (GUID) 在 query string 中添加 ?trace_id=xxx, 以返回 Trace ID 的内容。 注: 请把参数值设为 GUID 格式

返回的 HTTP 状态码：

状态码	含义	原因	解决办法
200	成功返回 JSON 主体	-	注：需检查 JSON 内容中的错误字段。如果该字段存在值，则表示发生了错误。
504	网关超时	请求未能成功到达目标服务器，且某个代理服务器发生了故障	• 重试 • 通知 PG SOFT

JSON 响应例子（仅供参考）：

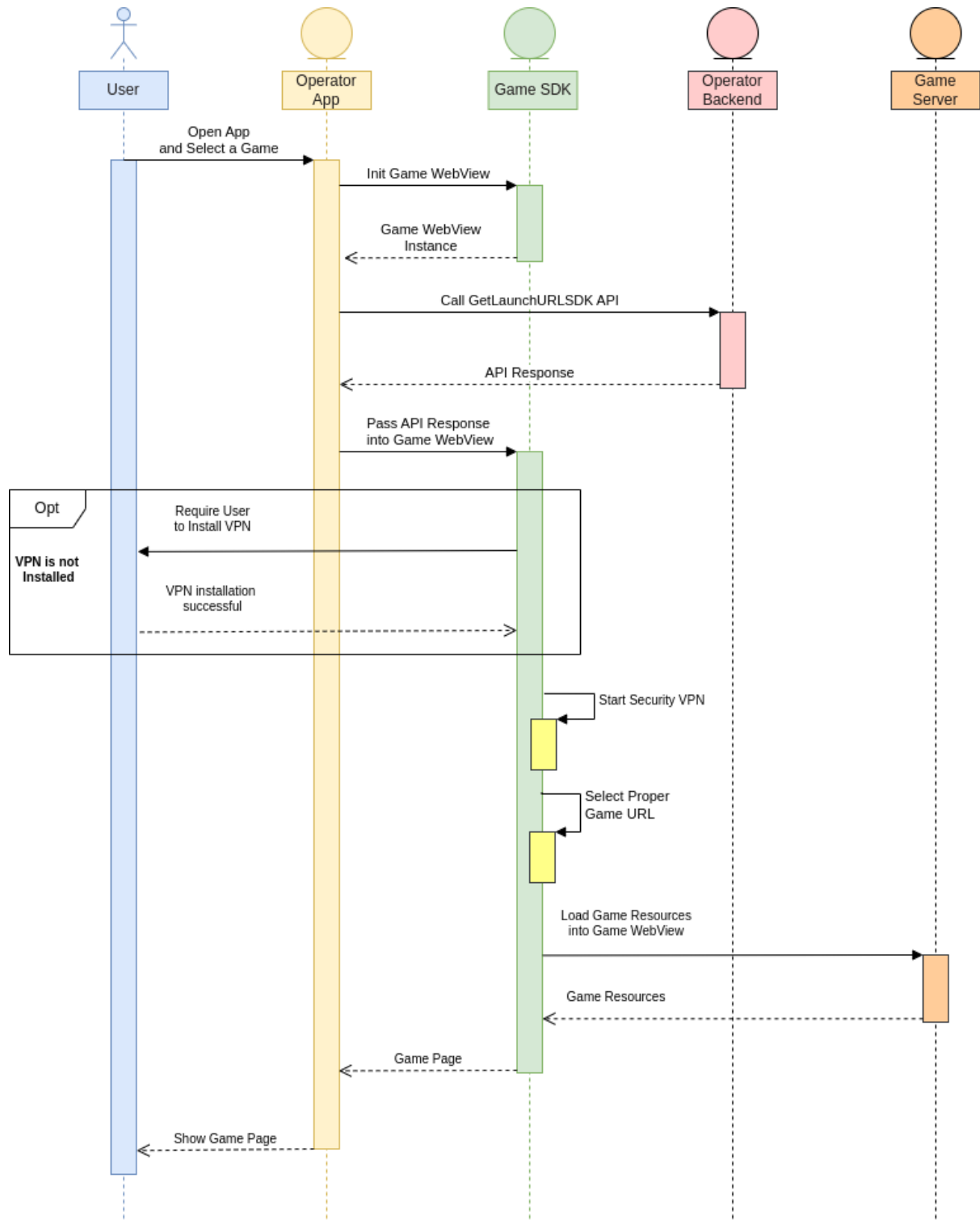
成功返回：

```
{
  "error": null,
  "version": "2",
  "data": "CAkQxzAD0odEi0OFisAR1kpRxwAHBkAZUZcMl0NGV8BEQJFDxZxChwyXAICHwAd..."
}
```

失败返回：

```
{
  "error": {
    "code": 400,
    "message": "Request validate error:\n\n\tclient_ip parse fail: 2400:9800:1f1:4039:1:0:3866",
    "tid": "xxx9988"
  }
}
```

3.1.3 iOS SDK Scheme 模式



3.1.3.1 实现方式

步骤 1：先决条件

1. 注册 Apple 开发者账户并加入 Apple 开发者计划。
2. 安装最新版 Xcode (15.0.1)
3. 向 PG SOFT 获取 PG SDK XCFramework。

注：

测试 SDK 文件：<https://app.pg-staging.com/game-sdk-ios/game-sdk.zip>

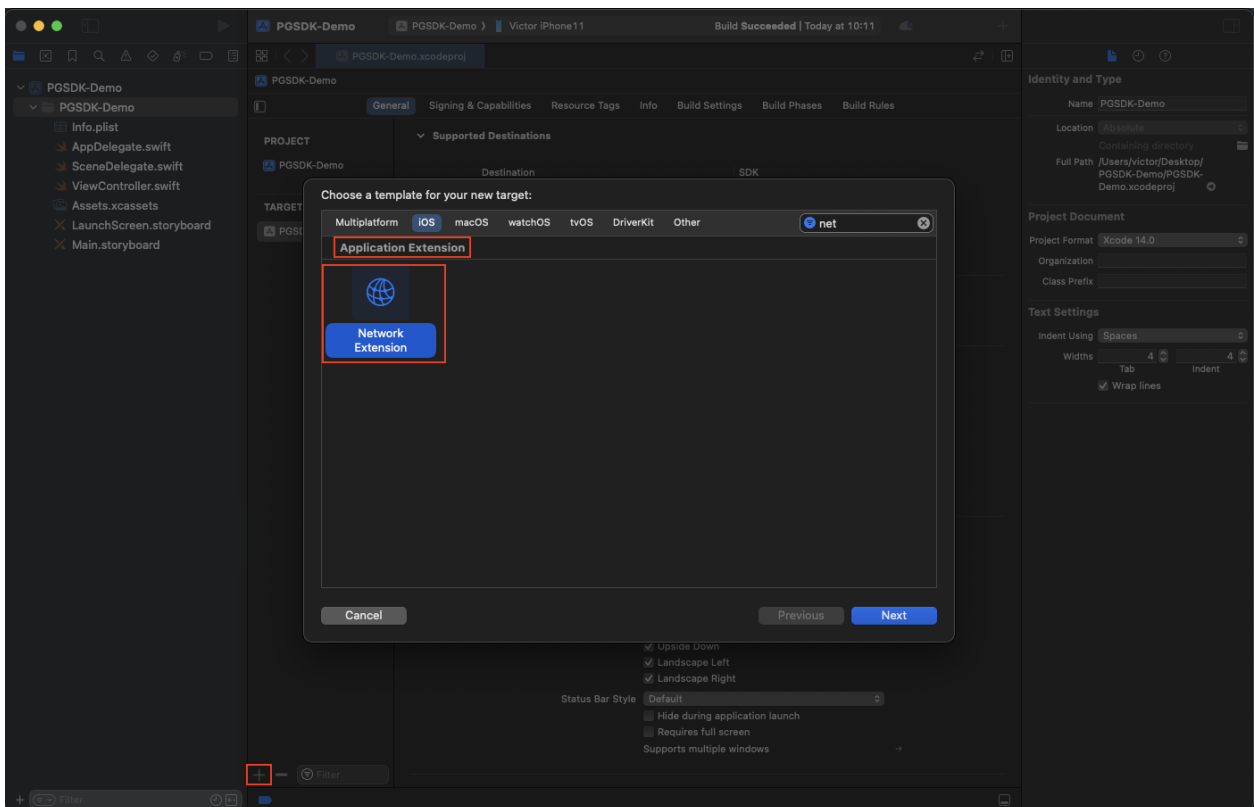
*注: 生产 SDK 文件请向 PG SOFT 索取。

步骤 2：PG SDK 对接步骤

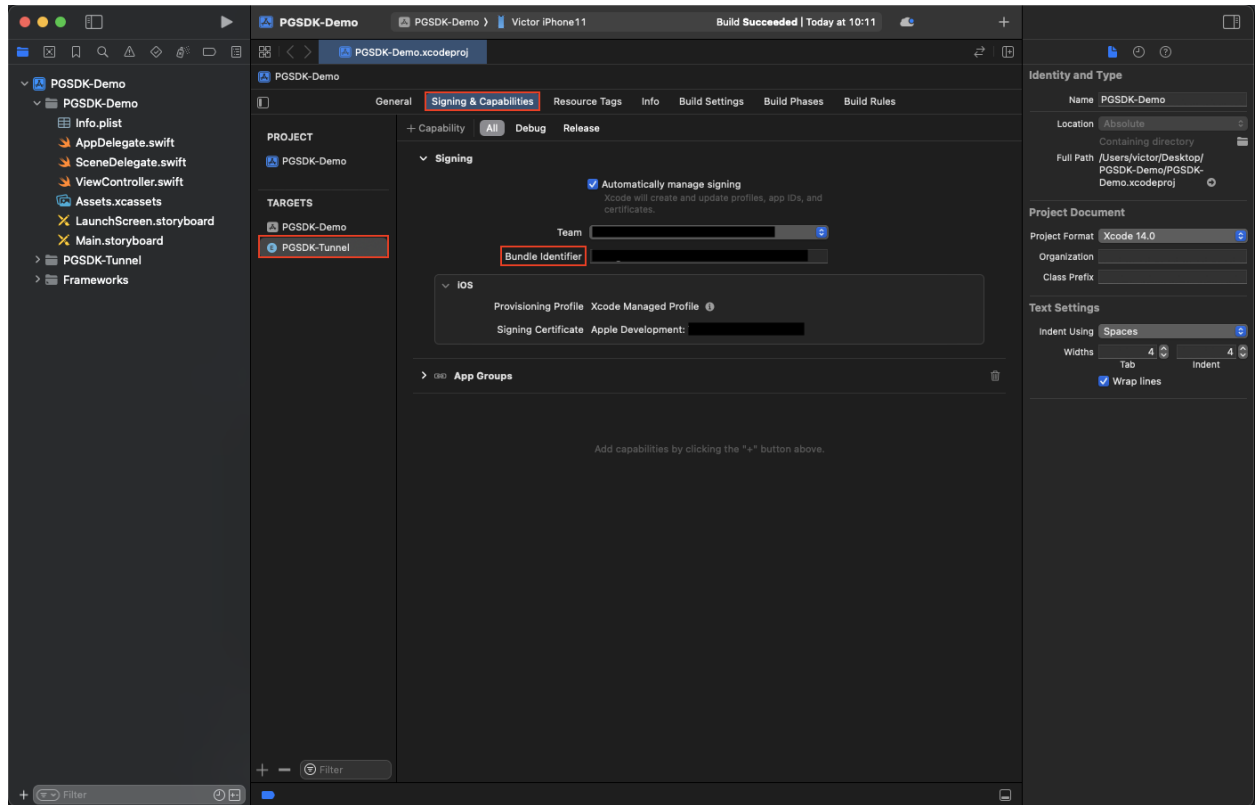
添加 Network Extension

添加一个 “Network Extension” target (Application Extension)

1. 点击 “+” 按钮以添加 Application Extension 的 “Network Extension” 至 iOS 项目中：



2. 在弹出窗口中输入“Product Name”（例如：XXX-Tunnel）并在“Provider Type”下拉选项中选择“Packet Tunnel”选项：



3. 请注意，后续将使用到 Application Extension 的 **“Bundle Identifier”**：

Choose options for your new target:

Product Name: PGSDK-Tunnel

Team: [Redacted] ▾

Organization Identifier: [Redacted]

Bundle Identifier: [Redacted].PGSDK-Tu...

Language: Swift ▾

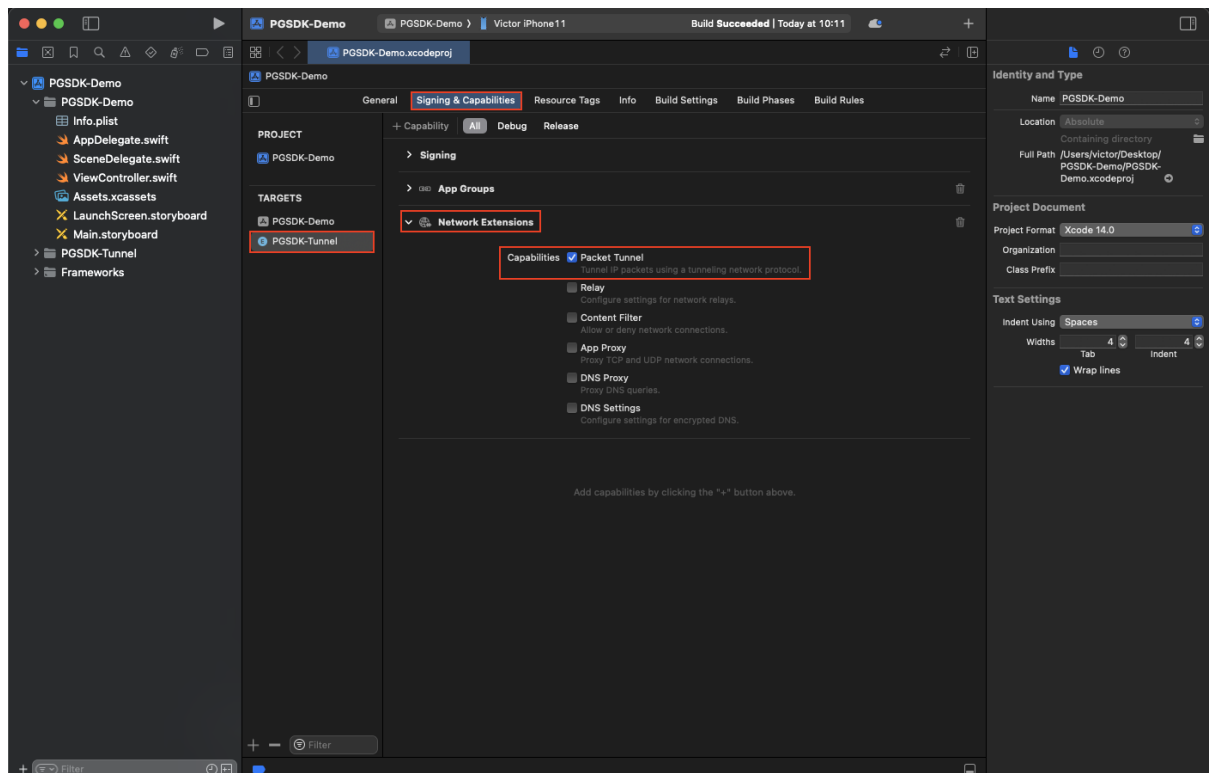
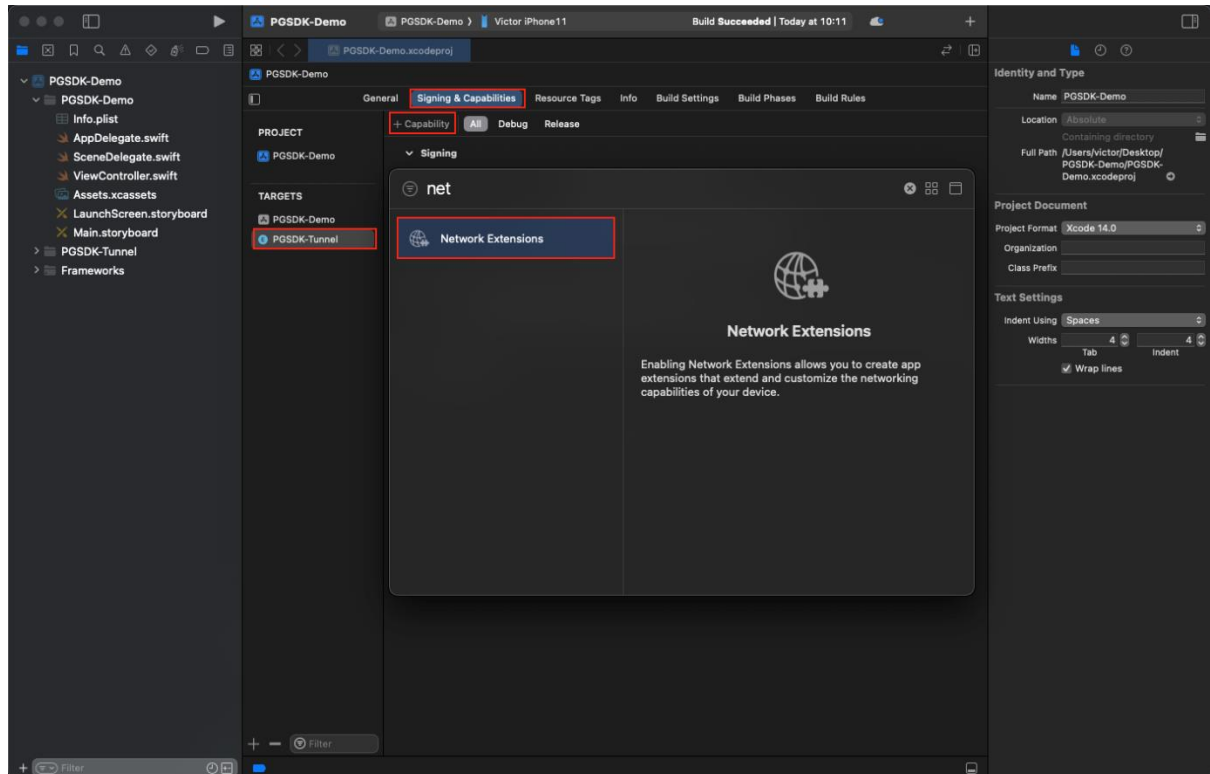
Provider Type: Packet Tunnel ▾

Project: PGSDK-Demo ▾

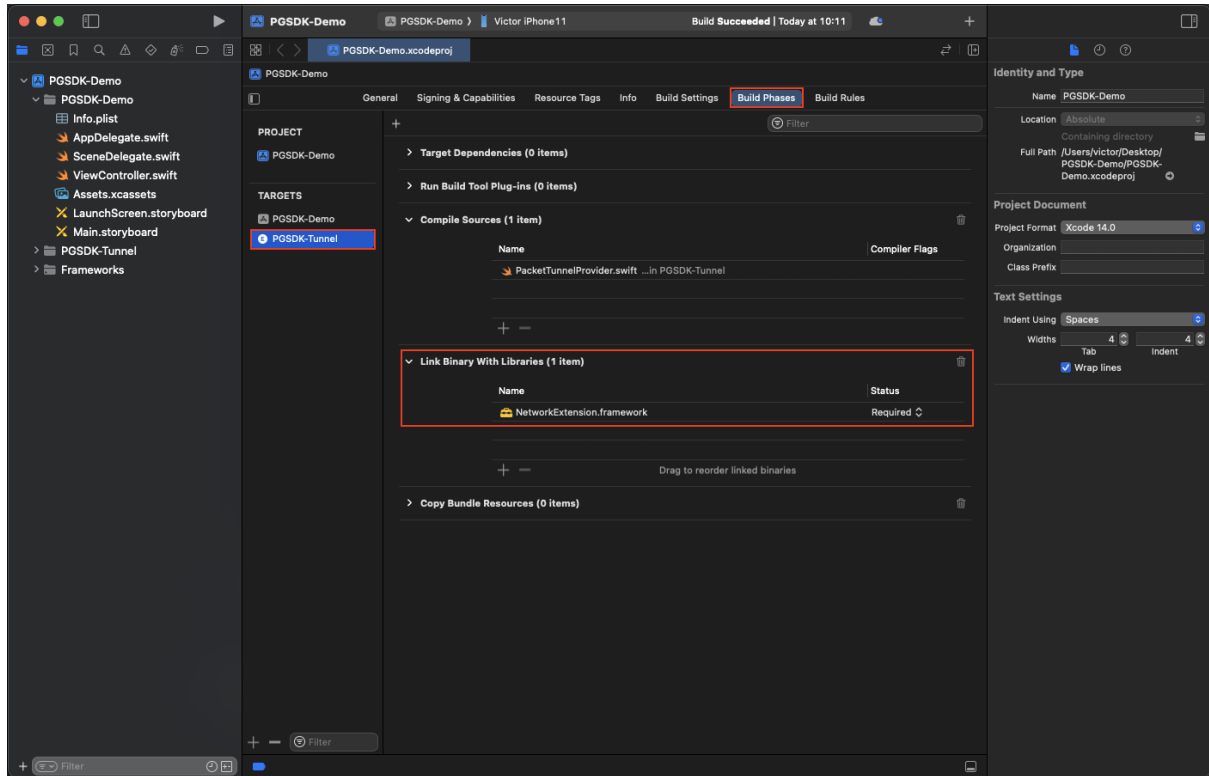
Embed in Application: PGSDK-Demo ▾

Cancel Previous Finish

4. 为上个步骤创建的“Application Extension”添加“Network Extensions”功能，并在“Network Extensions”部分勾选“Packet Tunnel”：

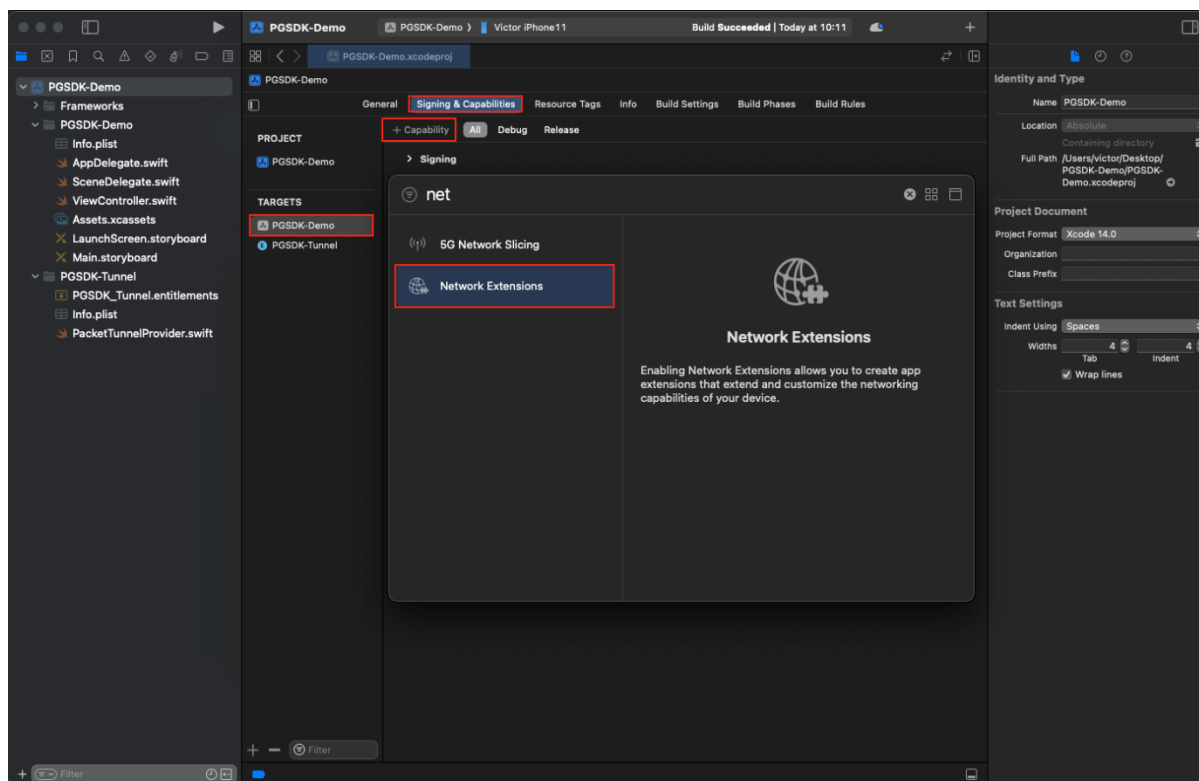


5. 在 “Application Extension” 的 “Build Phases” 分页中检查框架配置

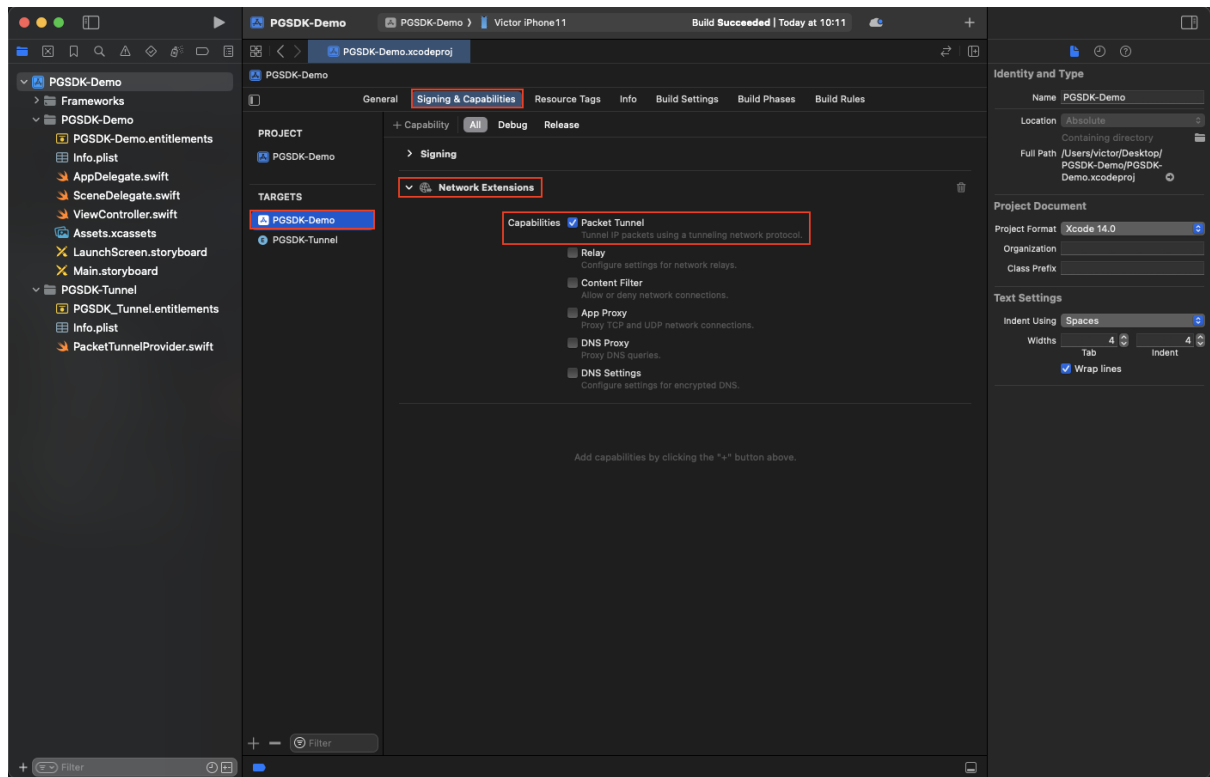


为主 target 添加 “Network Extensions” 功能

1. 选择主要 “target”，并跳转至 “Signing & Capabilities” 分页。点击 “+ Capability” 按钮以添加 “Network Extensions” 功能：

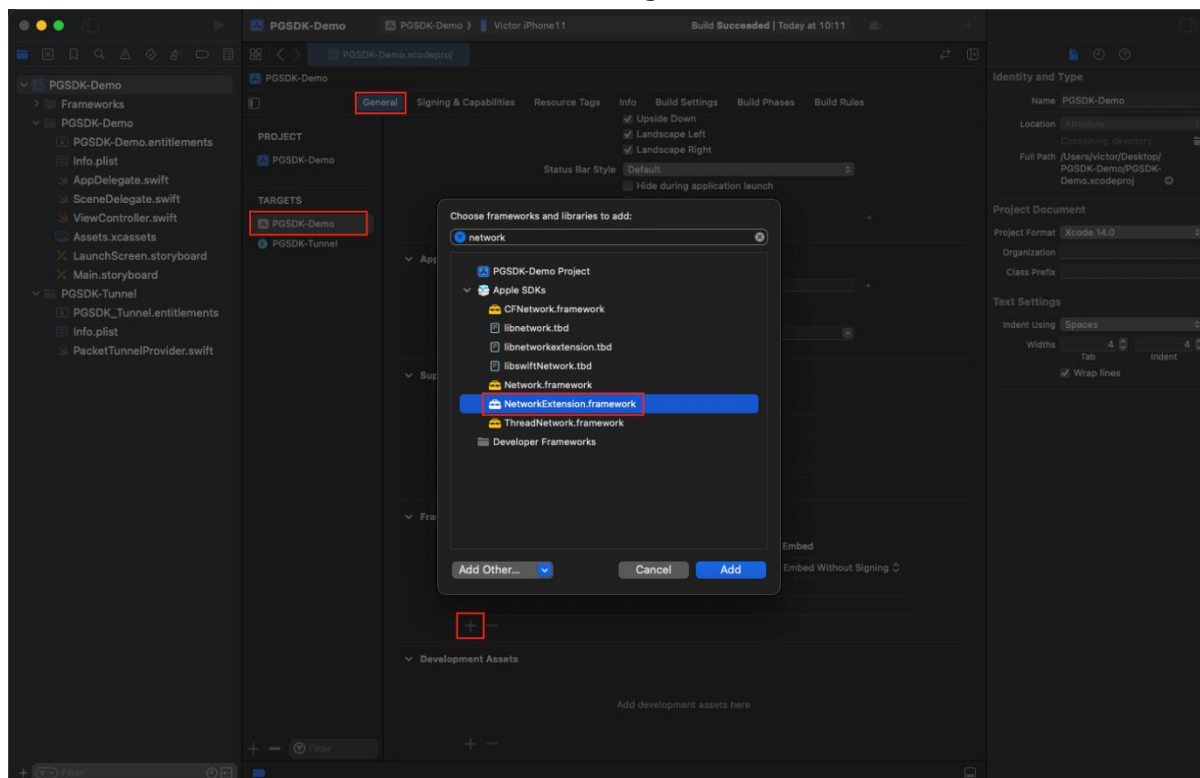


2. 勾选 “Network Extensions” 部分中的 “Packet Tunnel” 功能：

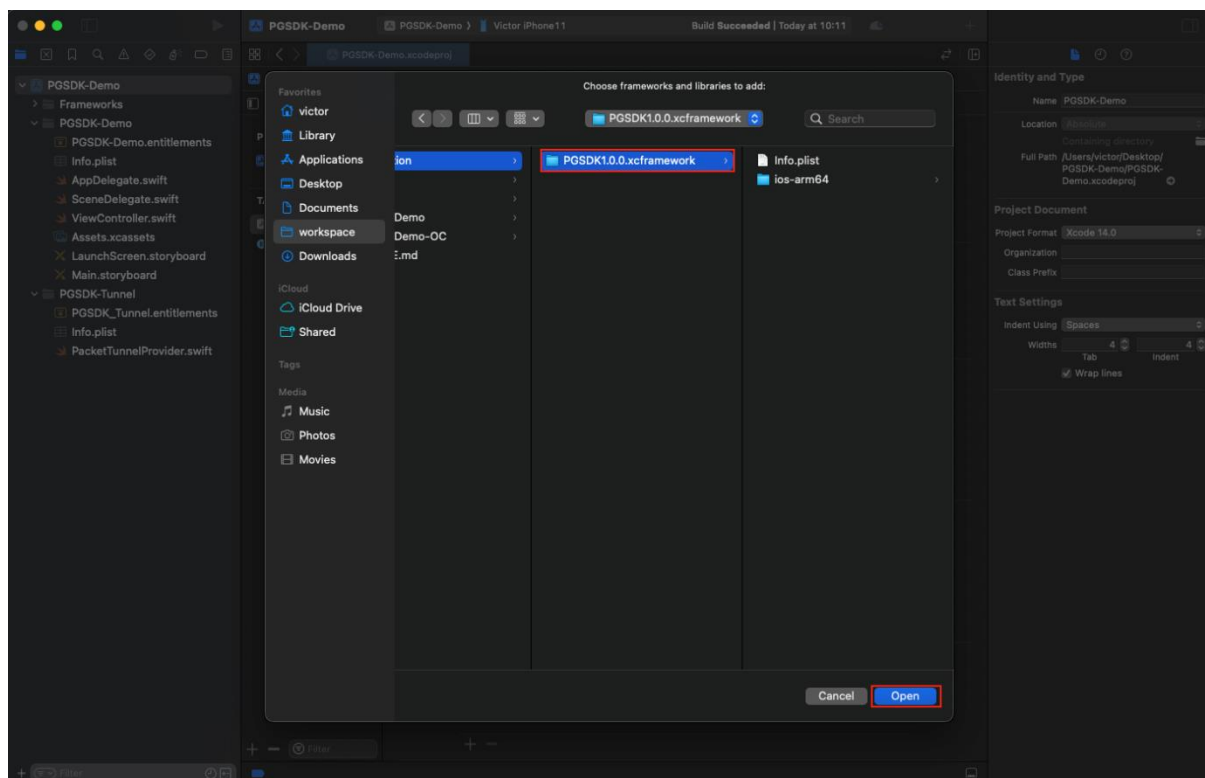
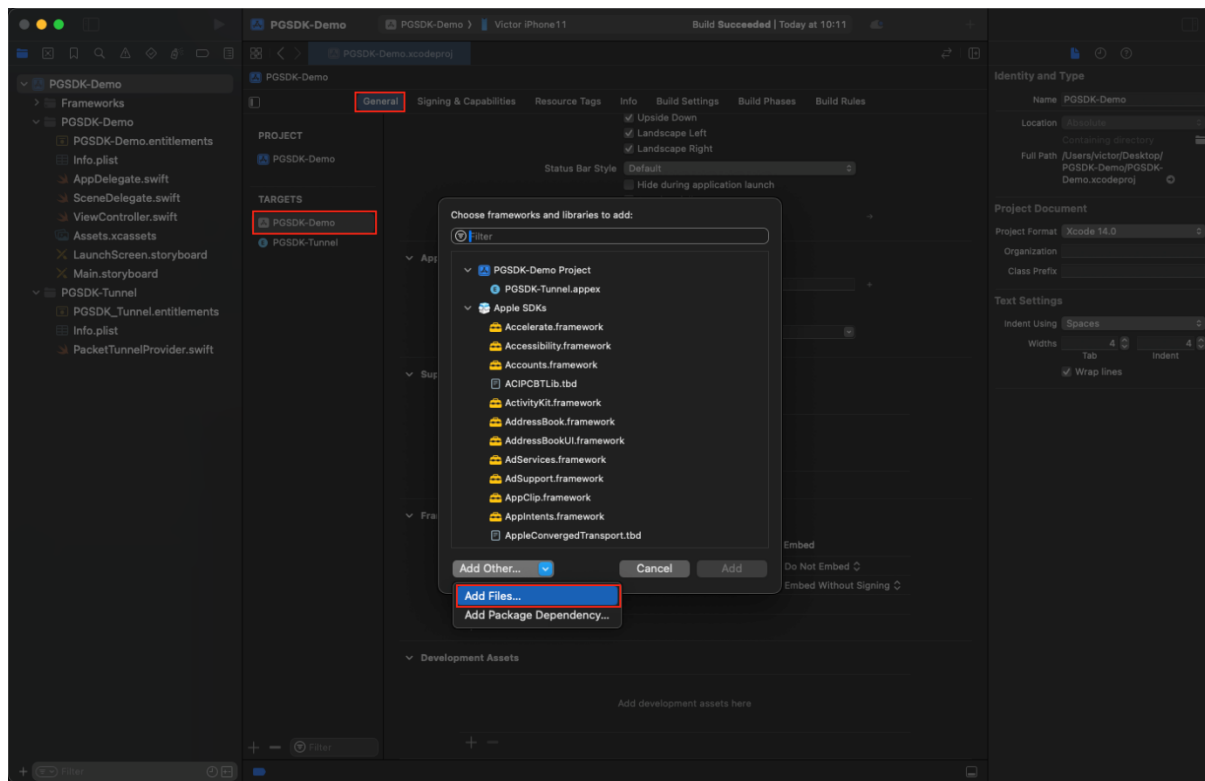


将框架添加至 “target”

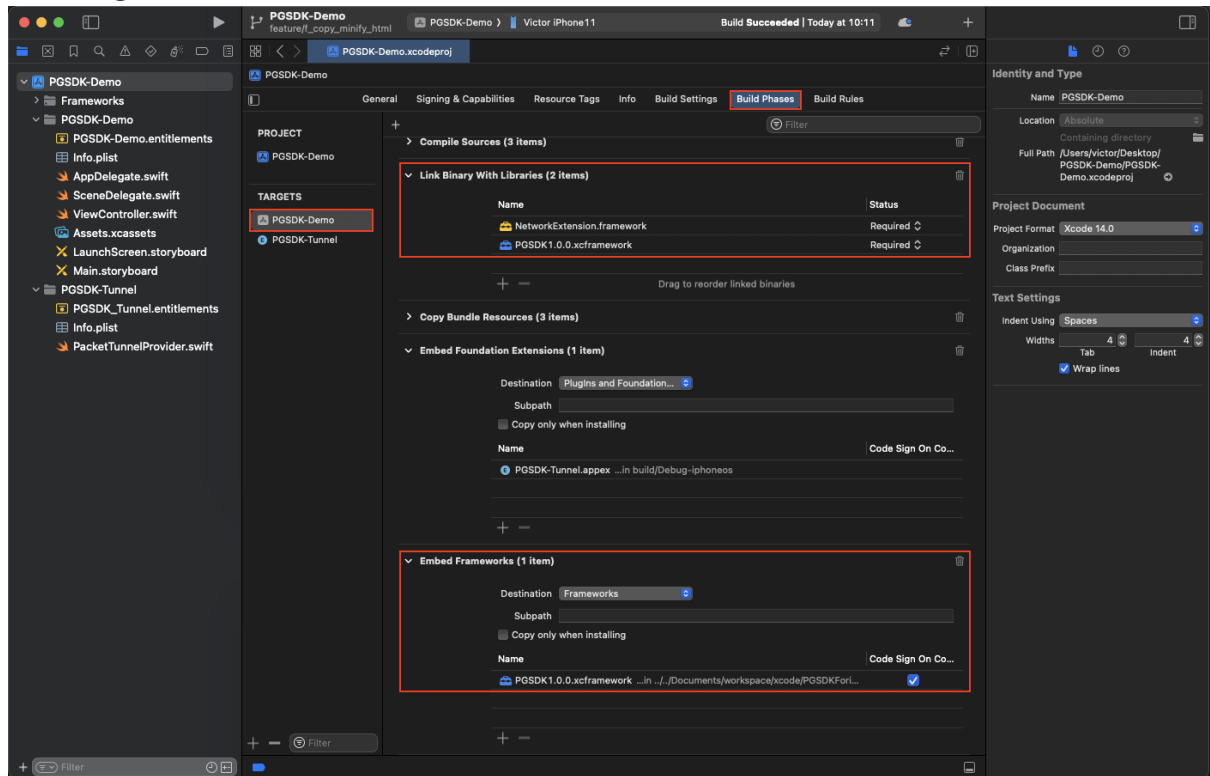
添加 “NetworkExtension.framework” 至主 target



添加 “PGSDK[x.x.x].xcframework” 至主 target



在主 target 的 “Build Phases” 分页中检查框架配置

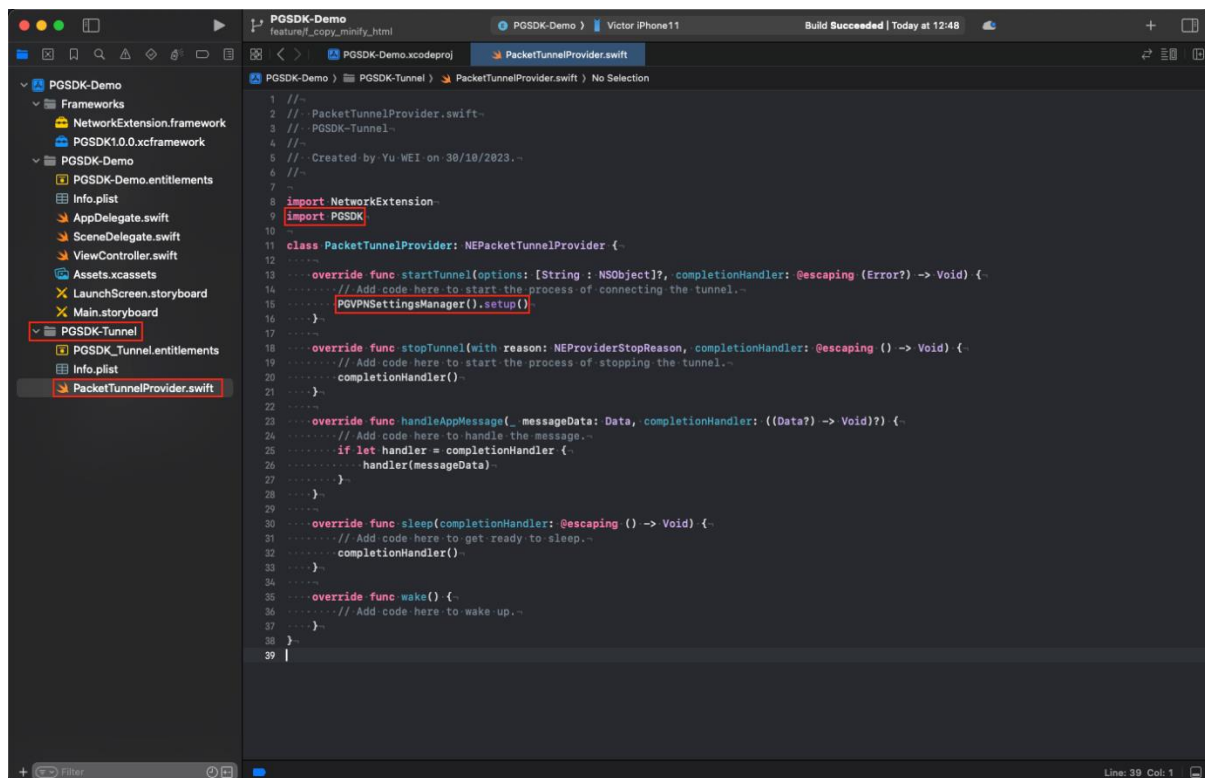


将 PGSDK 的 PacketTunnelProvider 对接至 Application Extension 中

1. 在 “Application Extension” 中打开 “PacketTunnelProvider.swift”
(Objective-C 中的 PacketTunnelProvider.h 和 PacketTunnelProvider.m)
2. 导入 “PGSDK” 并将以下代码添加到 “startTunnel” 函数中:

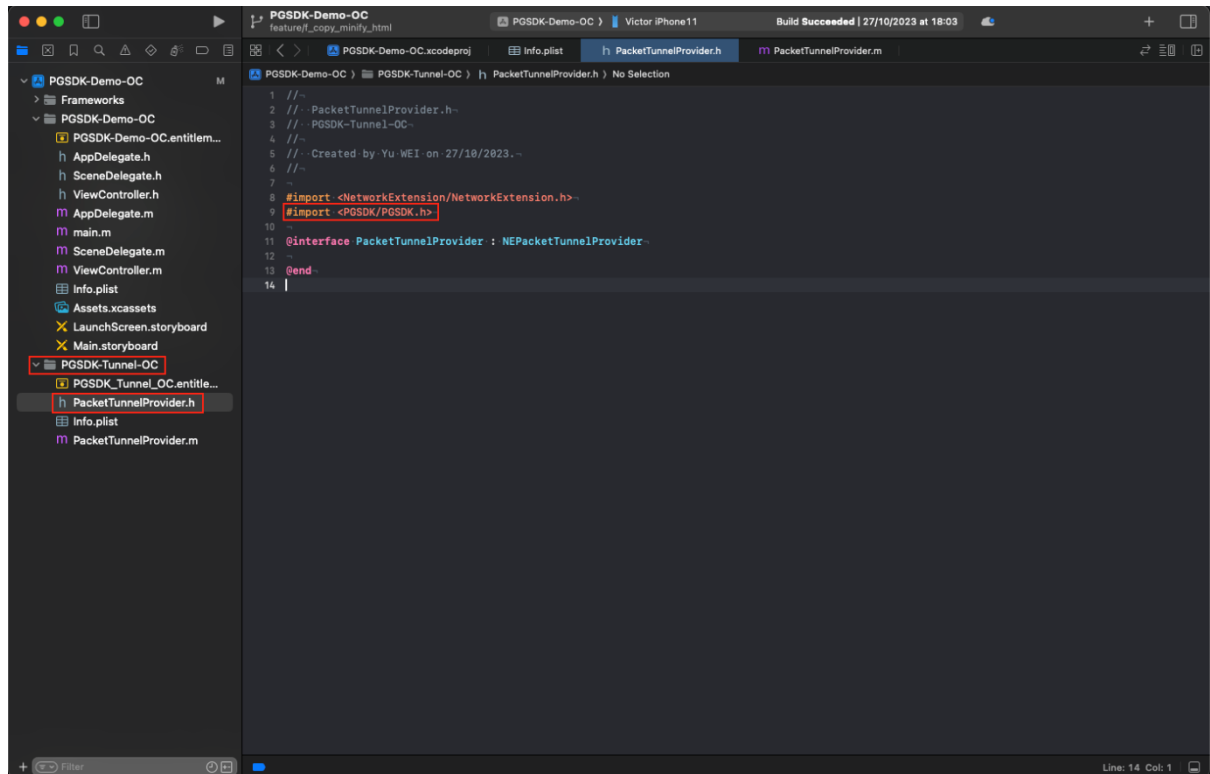
Swift:

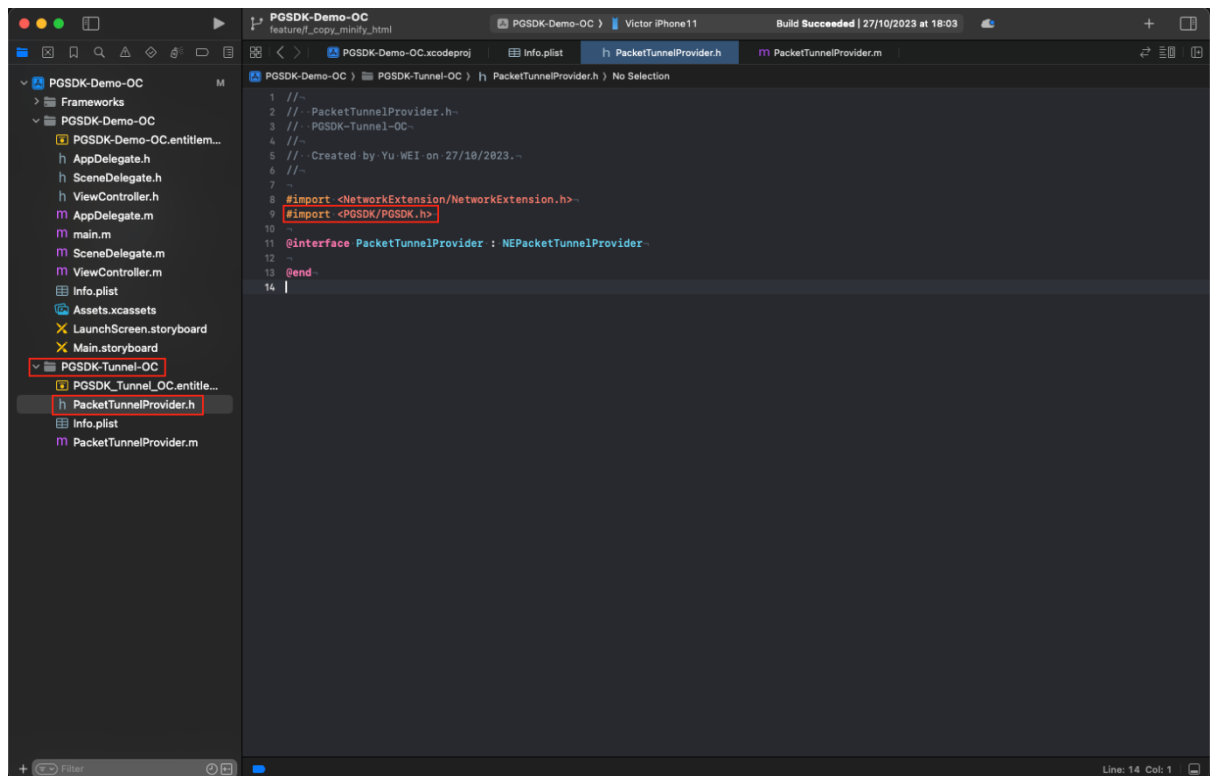
```
PGSoftVPNSettingsManager().setup()
```



Objective-C:

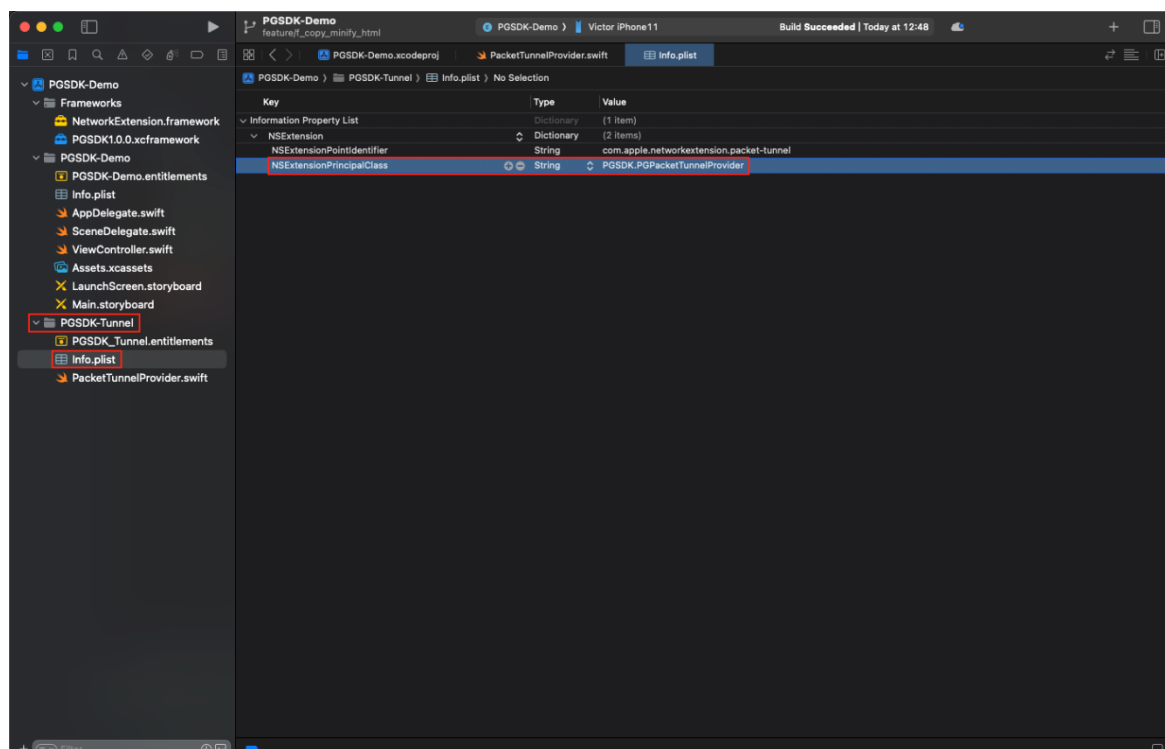
```
PGVPNSettingsManager *manager = [[PGVPNSettingsManager alloc] init];  
[manager setup];
```





3. 在 “Application Extension” 的 “**info.plist**” 中配置
 “**NSExtensionPrincipalClass**” :

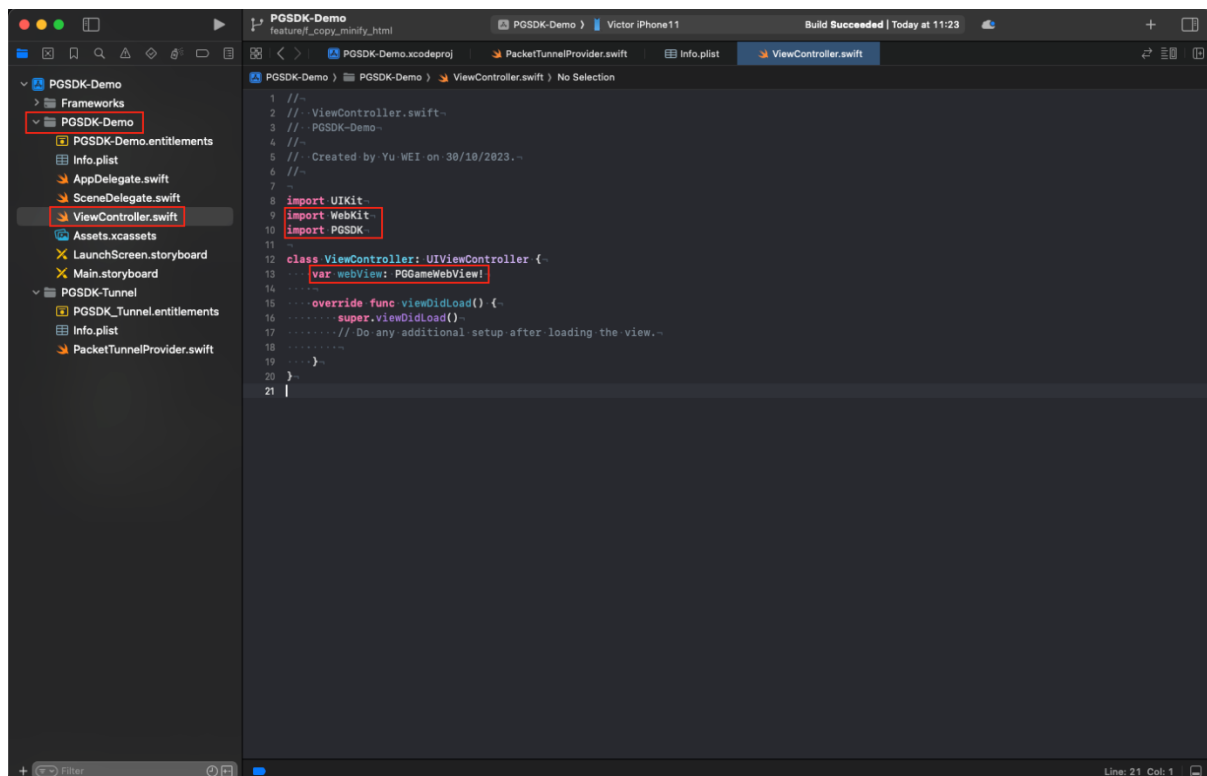
PGSoftSDK.PGSoftPacketTunnelProvider



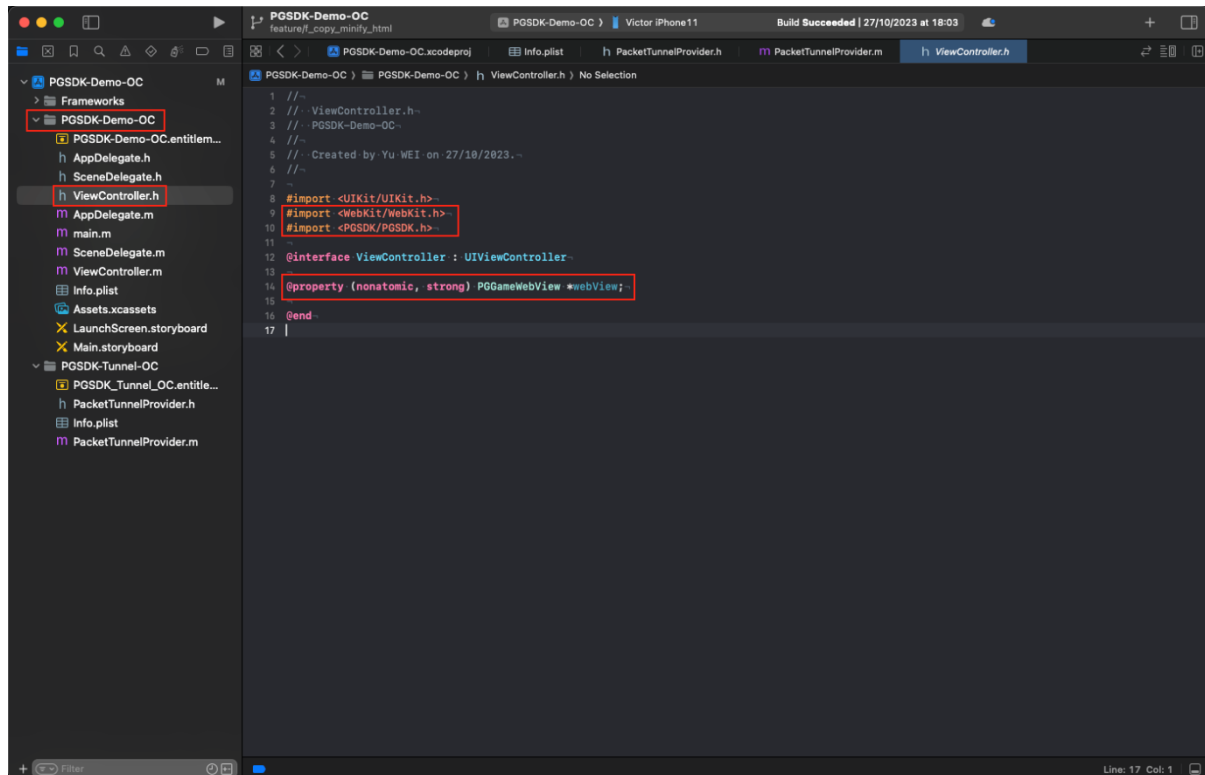
将 PGSoftSDK 的 PGSoftGameWebView 对接至 ViewController 中

1. 在主 target 中打开 “**ViewController.swift**” （Objective-C 中的 ViewController.h 和 ViewController.m）
2. 将 “**WebKit**” 和 “**PGSDK**” 导入至 “**ViewController**” 中，并添加类型为 “**PGSoftGameWebView**” 的变量：

Swift:



Objective-C:



3. 在 “viewDidLoad” 函数中初始化 “PGSoftGameWebView”

请将 2.1.1-3 部分的 (Bundle Identifier) 值传递至 “providerBundleIdentifier” 参数:

请查阅 API 参考¹⁸以了解 “dnsType” 。

Swift:

```

self.webView = PGGameWebView(frame: self.view.bounds,
                             configuration: WKWebViewConfiguration(),
                             providerBundleIdentifier: "", // The value of
section 2.3.2.1.1-3
                             dnsType: PGDNSConfigType.google)
if let parentView = self.webView.superview {
    self.webView.frame = parentView.safeAreaLayoutGuide.layoutFrame
}
self.view.addSubview(self.webView)
  
```

¹⁸ 请参考 [API 参考](#)部分以获取更多信息

Objective-C:

```
self.webView = [[PGGameWebView alloc] initWithFrame:self.view.bounds
                                                    configuration:[WKWebViewConfiguration
new]
                                                    providerBundleIdentifier:@" " // The value of
section 2.3.2.1.1-3
                                                    dnsType:PGDNSConfigTypeGoogle];
UIView *parentView = self.webView.superview;
if (parentView) {
    self.webView.frame = parentView.safeAreaLayoutGuide.layoutFrame;
}
[self.view addSubview:self.webView];
```

4. 调用 “GetLaunchURLSDK” API¹⁹并加载该响应。请调用 PGGameWebView 实例的 “load” 函数，并将 API 响应以字符串传递：

Swift:

```
let apiResp = " " // Response of GetLaunchURLSDK API
self.webView.load(apiResp)
```

Objective-C:

```
NSString *apiResp = @" "; // Response of GetLaunchURLSDK API
[self.webView load:apiResp];
```

¹⁹ 请参考 [GetLaunchURLSDK API](#) 部分以获取更多信息

5. 以下为“ViewController”的完整代码供参考。

Swift:

```
import UIKit
import WebKit
import PGSDK

class ViewController: UIViewController {
    var webView: PGGameWebView!

    override func viewDidLoad() {
        super.viewDidLoad()
        // Do any additional setup after loading the view.

        self.webView = PGGameWebView(frame: self.view.bounds,
                                       configuration:
WKWebViewConfiguration(),
                                       providerBundleIdentifier: "", // The
value of section 2.3.2.1.1-3
                                       dnsType: PGDNSConfigType.google)
        if let parentView = self.webView.superview {
            self.webView.frame = parentView.safeAreaLayoutGuide.layoutFrame
        }
        self.view.addSubview(self.webView)

        let apiResp = "" // Response of GetLaunchURLSDK API
        self.webView.load(apiResp)
    }
}
```

Objective-C:

```
#import <UIKit/UIKit.h>
#import <WebKit/WebKit.h>
#import <PGSDK/PGSDK.h>

@interface ViewController : UIViewController

@property (nonatomic, strong) PGGameWebView *webView;

@end
```

```
#import "ViewController.h"

@interface ViewController ()

@end

@implementation ViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    // Do any additional setup after loading the view.

    self.webView = [[PGGameWebView alloc] initWithFrame:self.view.bounds
configuration:[WKWebViewConfiguration new]
               providerBundleIdentifier:@""] // The value of
section 2.3.2.1.1-3

dnsType:PGDNSConfigTypeGoogle];
    UIView *parentView = self.webView.superview;
    if (parentView) {
        self.webView.frame = parentView.safeAreaLayoutGuide.layoutFrame;
    }
    [self.view addSubview:self.webView];

    NSString *apiResp = @""; // Response of GetLaunchURLSDK API
    [self.webView load:apiResp];
}

@end
```

3.1.4 API 参考

3.1.4.1 实现方式

PGDNSConfig

```
/**
 * Initialize a PGDNSConfig instance
 *
 * PGSDK provides custom DNS functionality, passing necessary parameters to
 * create PGDNSConfig instance
 *
 * @param {String} dnsName - DNS name (e.g. Google, Cloudflare)
 * @param {String} serverURL - DNS over HTTPS resolver URL
 * @param {String} apiURL - DNS over HTTPS JSON API URL
 * @param {[String]?} servers - Optional, DNS server IP (IPv4 or IPv6)
 * @returns {PGDNSConfig} PGDNSConfig instance
 */
init(dnsName: String, serverURL: String, apiURL: String, servers: [String]?)
```

PGDNSConfigType

```
/**
 * Enumeration type of DNS Type
 *
 * PGSDK provides 4 preconfigured DNS configuration (Google, Cloudflare,
 * Tencent, Alibaba). You can also use a "custom" type, in which case you need to
 * initialize a PGDNSConfig instance yourself
 */
enum PGDNSConfigType {
    case google
    case cloudflare
    case tencent
    case ali
    case custom
}
```

PGSDKDevToolKit

```
/**
 * Get PGSDK version string
 */
static func getVersionString() -> String?
```

PGGameWebView

```
/**
 * Initialize a PGGameWebView instance
 *
 * Initialize the PGGameWebView instance and use the preconfigured DNS
 configuration
 *
 * @param {CGRect} frame - Foundation CGRect
 * @param {WKWebViewConfiguration} configuration - WebKit
 WKWebViewConfiguration
 * @param {String} providerBundleIdentifier - Application Extension Bundle
 Identifier
 * @param {PGDNSConfigType} dnsType - Enumeration PGDNSConfigType
 * @returns {PGGameWebView} PGGameWebView instance
 */
init(frame: CGRect, configuration: WKWebViewConfiguration,
providerBundleIdentifier: String, dnsType: PGSDK.PGDNSConfigType)
```

```
/**
 * Initialize a PGGameWebView instance
 *
 * Initialize the PGGameWebView instance and use custom DNS configuration
 *
 * @param {CGRect} frame - Foundation CGRect
 * @param {WKWebViewConfiguration} configuration - WebKit
 WKWebViewConfiguration
 * @param {String} providerBundleIdentifier - Application Extension Bundle
 Identifier
 * @param {PGDNSConfig} dnsConfig - PGDNSConfig instance
 * @returns {PGGameWebView} PGGameWebView instance
 */
init(frame: CGRect, configuration: WKWebViewConfiguration,
providerBundleIdentifier: String, dnsConfig: PGSDK.PGDNSConfig)
```

```
/**
 * Load the game based on the "GetLaunchURLSDK" API response
 *
 * After initializing the PGGameWebView instance and getting the response of
 the "GetLaunchURLSDK" API, pass the response as a string to the "load"
 function of the PGGameWebView instance to load the game
 *
 * @param {String} responseString - API response
 */
func load(_ responseString: String)
```

PGVPNSettingsManager

```
/**  
 * Initialize a PGVPNSettingsManager instance  
 */  
init()
```

```
/**  
 * Perform necessary settings  
 */  
func setup()
```

3.1.5 其他

1. PGSDK 支持的最低 iOS 版本为 14.0。
2. 由于项目涉及了 iOS 模拟器不支持的 “Network Extension” 框架，请在实体设备上
进行运行或调试。

3.2 请求示例

3.2.1 直接启动 HTML 游戏模式

以下为运营商前端的 HTML 代码例子。玩家通过点击游戏启动链接以启动 PG SOFT 游戏：

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
</head>
<body><a href="https://<Operator Backend API>/pg-game-launcher-html-
mode.php?param1=xxx&param2=yyy" target="_blank">Launch Game</a></body>
</html>
```

1. 玩家访问运营商网站并点击 PG SOFT 游戏。运营商前段将向后端服务发出请求以获取 HTML 代码。
2. 后端服务接收请求后，将调用 PG SOFT 的 GetLaunchURLHTML API。
3. PG SOFT 的 GetLaunchURLHTML API 返回 HTML 代码。
4. 运营商后端 API 将 PG SOFT 返回的 HTTP 标头和 HTML 代码原样传递给运营商前端（即步骤 1 中打开的新窗口），不得进行任何修改。
5. 前端接收到 HTML 代码后，在玩家浏览器上执行并展示 HTML 内容（启动 PG SOFT 游戏）。

3.2.2 iframe 模式 (推荐)

注:

使用 iframe 模式有助于保护 PG SOFT 的域名安全，特别推荐在域名被劫持或封锁风险较高的地区使用该方式。

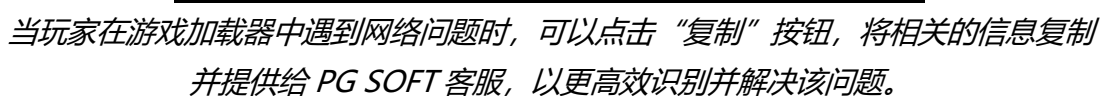
```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head></head>
<body>
  <iframe allow="web-share *; clipboard-write *; screen-wake-lock *; fullscreen
*" src="https://<Operator Backend API>/pg-game-launcher-html-
mode.php?param1=xxx&param2=yyy">
  </iframe>
</body>
</html>
```

1. 玩家访问运营商网站并点击 PG SOFT 游戏。运营商前段将向后端服务发出请求以获取 HTML 代码。
2. 后端服务接收请求后，将调用 PG SOFT 的 GetLaunchURLHTML API。
3. PG SOFT 的 GetLaunchURLHTML API 返回 HTML 代码。
4. 运营商后端 API 将 PG SOFT 返回的 HTTP 标头和 HTML 代码原样传递给运营商前端（即步骤 1 中打开的新窗口），不得进行任何修改。
5. 前端接收到 HTML 代码后，在玩家浏览器上执行并展示 HTML 内容（启动 PG SOFT 游戏）。

注:

为确保 PG SOFT 游戏能够正常启动，在使用 iframe 模式时，必须在 iframe 标签中加入允许属性。

- Clipboard-write:





若玩家无法直接打开 PG SOFT 的验证页面，也可通过复制验证链接以手动进行验证。

- Fullscreen 和 screen-wake-lock:
游戏引擎正常运行必需。

3.2.3 Ajax 模式（不推荐）

*注：Ajax 模式可能会面临以下潜在问题，因此并不推荐使用：

- 情境 1：在 iOS 的 Safari 浏览器中，会出现一个“屏蔽弹出窗口”提示，表明弹出窗口被禁止。用户需点击“始终允许”，然后再点击“打开”以正常启动。（对应下方示例代码中的方法 1）
- 情境 2：Safari 浏览器的“返回”按钮将无法返回到启动 PG SOFT 游戏前的页面。（对应下方示例代码中的方法 2）

例子：

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.4/jquery.min.js"></script>
<script>
$(document).ready(function(){
  $("button").click(function(){
    $.getJSON("https://httpbin.org/anything", { html: '<h1>New
Window</h1><p>Hello World!</p>' }, function(data) {
      var html = data.args.html;

      // Method 1, use new window to execute HTML
      var newWindow = window.open('', '_blank');
      newWindow.document.open();
      newWindow.document.write(html);
      newWindow.document.close();

      // Method 2, use existing window to execute HTML.
      document.write(html);
    });
  });
});
</script>
</head>
<body><button>Launch Game</button></body>
</html>
```

1. 玩家点击“Launch Game”按钮时，将通过 Ajax 触发运营商后端 API。
2. 运营商的后端 API 将向 PG SOFT 的 GetLaunchURLHTML API 发送请求。
3. PG SOFT 的 GetLaunchURLHTML API 返回 HTML 代码。

4. 运营商后端 API 将 PG SOFT 返回的 HTTP 标头和 HTML 代码原样传递给运营商前端（即步骤 1 中打开的新窗口），不得进行任何修改。
5. 在收到运营商的后端 API 响应后，Ajax 将执行回调函数，并通过浏览器执行 HTML 代码（启动 PG SOFT 游戏）。

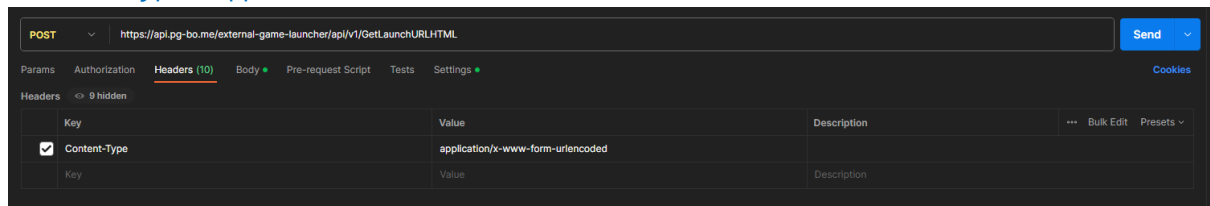
3.2.4 POSTMAN (RAW 主体)

以下为通过 Postman 调用 PG SOFT GetLaunchURLHTML API 获取的 HTML 代码示例。

请求参数：

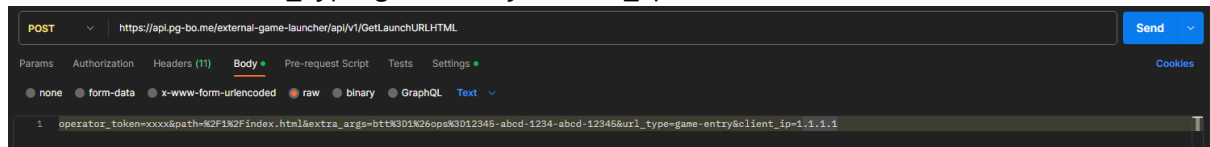
标头：

Content-Type: application/x-www-form-urlencoded



主体参数(raw)：

operator_token=xxxx&path=%2F1%2Findex.html&extra_args=btt%3D1%26ops%3D12345-abcd-1234-abcd-12345&url_type=game-entry&client_ip=1.1.1.1



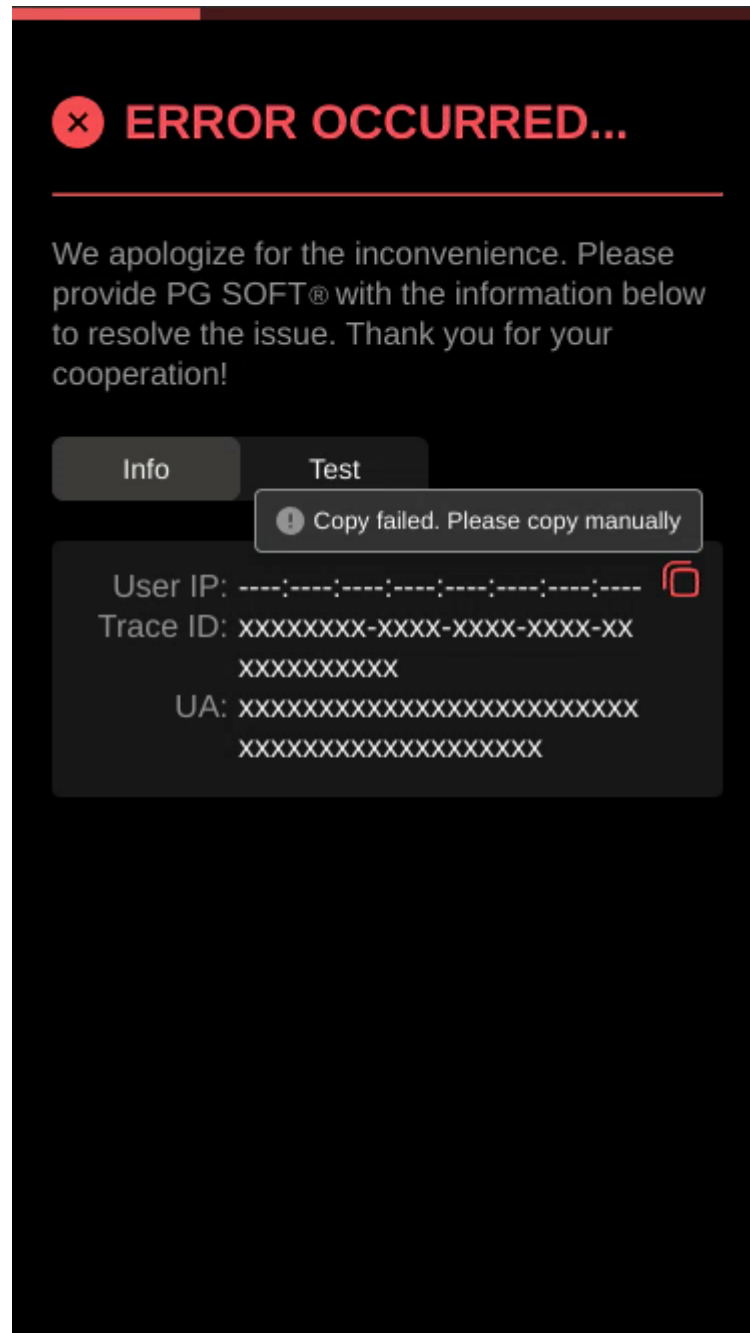
3.2.5 CURL command

以下为通过 CURL command 调用 PG SOFT GetLaunchURLHTML API 获取的 HTML 代码示例。

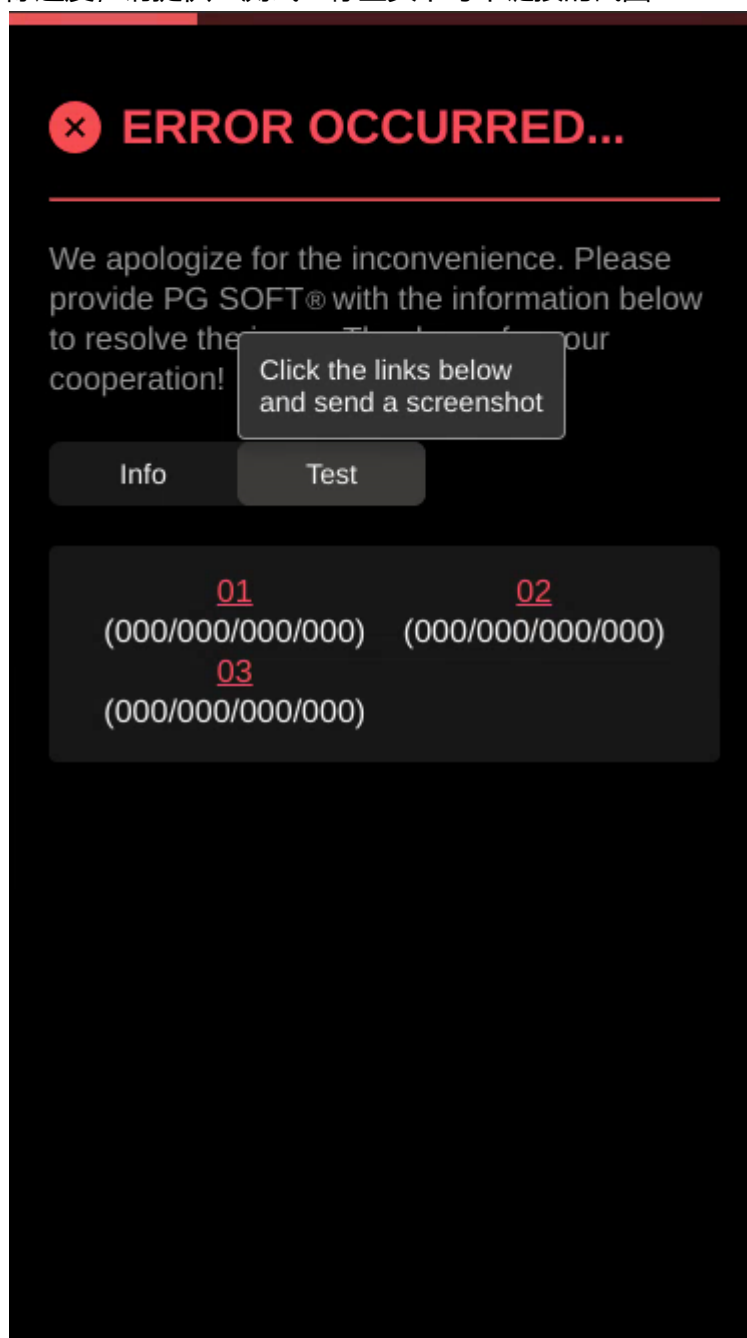
请求参数:

```
curl -v -X POST "https://api.pg-bo.me/external-game-launcher/api/v1/GetLaunchURLHTML"  
-H "Content-Type: application/x-www-form-urlencoded"  
-d  
"operator_token=xxxx&path=%2F1%2Findex.html&extra_args=btt%3D1%26ops%3D12345-  
abcd-1234-abcd-12345&url_type=game-entry&client_ip=1.1.1.1"
```


2. 若浏览器不支持复制操作，页面中的折叠内容将自动展开。在该情况下，请提供如下截图：



3. 为加快故障排除速度，请提供“测试”标签页下每个链接的截图：



3.3 游戏启动限制

PG SOFT 游戏不支援一个账号多重登录或一个玩家同时打开多个游戏。以上操作将导致玩家被锁定，同时将被踢出游戏及报错：

S1302XXXXXXXX: 未能成功登录认证

S1309XXXXXXXX: 玩家账号被冻结

S1305XXXXXXXX: 服务器请求失败

若玩家在运行游戏时遇到以上错误，只需关闭其它游戏窗口并重启游戏，即可解锁他们的账号：

S1302XXXXXXXX: 请直接重新启动游戏

S1309XXXXXXXX: 玩家将与 10 分钟后解锁，请于 10 分钟后重新启动游戏

S1305XXXXXXXX: 请直接重新启动游戏

3.4 常见问题

1. 调用 game-launcher API 返回 403 错误
访问 game-launcher API 需进行 API 白名单设置。请联系 PG SOFT 客服进行白名单设置。
2. DNS 错误、网络异常或连接超时
请检查玩家的网络设置。
3. HTTP 400/404 错误
请检查请求的 Content-Type。
4. HTTP 500 错误
请重试请求。若问题仍存在，请联系 PG SOFT 以获取进一步协助。
5. 页面显示异常字符：
 - a. 请确认浏览器已启用 gzip 压缩功能。
 - b. 检查营运商服务器是否正确传递了 Content-Encoding HTTP 标头。
6. 浏览器脚本报错
如需了解支持的浏览器类型与版本，请联系 PG SOFT 客服。

4 启动游戏 JS 库

运营商需使用 PG SOFT JS 库启动游戏，以便为玩家提供最佳的游戏体验：

- 根据用户的显示器大小而自动调整弹出窗口。
- PG SOFT 游戏均设计为竖屏显示，弹出窗口的格式与产品设计一致。
- 实现最佳的游戏界面高度，并符合用户使用习惯，提升用户的感官享受。
- 向用户展示最高的游戏视觉质量，提供最佳的游戏体验。

4.1 避免浏览器弹出窗口受阻挡

弹出窗口被阻挡时所涉及的一般规则是：

- 从 JavaScript 调用，而不是直接从用户操作所调用。
- 弹出内容并不是源于同一个域名。

建议运营商遵循以下说明以避免浏览器弹出窗口受阻挡：

- 创建前端页面并调用自己的后端服务（HTML Scheme²⁰）
- 当玩家点击游戏启动按钮或链接时，使用 PG SOFT JS 功能（PGSDK.launchGame）并启动运营商前端页面。

²⁰ 请参考 [HTML Scheme 方式](#) 的部分以获取更多信息

4.2 实现方式

步骤 1

添加以下代码至 HTML <head> 标签:

```
<script defer src="https://public.pg-staging.com/sdk/main.min.js"></script>
```

注:

运营商也可以自行托管 JS 库: <https://public.pg-staging.com/sdk/main.min.js>

步骤 2

运营商需要创建自行托管的前端页面以调用自己的后端服务 (HTML Scheme²¹) 以获得启动游戏的 HTML 代码。

运营商前端页面的代码示例:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.4/jquery.min.js">
</script>
<script>
    $(document).ready(function(){
        $.post( "https://<Operator backend service>/api",
            { param1: "value1", param2: "value2" },
            function(data) {
                window.document.write(data); //Execute HTML Code
            });
    });
</script>
</head>
</html>
```

²¹ 请参考 [HTML Scheme 方式](#) 的部分以获取更多信息

步骤 3

使用以下 JS script 以启动 PG SOFT 游戏。

```
PGSDK.launchGame('{operator_ frontend _URL}');
```

参数:

参数名称	参数类型	必需项	描述
operator_frontend_URL	String	是	运营商于步骤 2 里自行托管的前端页面将调用自身的后端服务

例子:

```
PGSDK.launchGame('https://[OperatorDomain]/operator-frontend.html');
```


5 对接 API

5.1 运营商提供对接 API

5.1.1 API 格式

5.1.1.1 请求参数

该 API 使用 HTTP 表单方式以及 RESTful 终点架构。PG SOFT 将根据以下内容格式发送请求：
`Content-Type: application/x-www-form-urlencoded`

5.1.1.2 返回结果

API 返回 JSON 格式的返回结果。对于成功和失败的 API 请求，运营商需返回 HTTP 状态码为 200 的 API 响应结果。

为避免请求被判定为超时，API 响应需在 10 秒内返回。

API 对于全部 API 方式都使用全球 JSON 格式的返回结果，其内容类型如下：
`Content-Type: application/json`

参数:

参数名称	参数类型	描述
data ²²	JSON Object	API 方式的返回结果信息
error	JSON Object	发生错误或异常时的错误信息

例子:

成功返回

```
{
  "data": {
    [API 返回信息。JSON 格式将根据各个 API 而有所不同]
  },
  "error": null,
}
```

失败/错误返回

```
{
  "data": null,
  "error": {
    "code": "[ 错误代码 ]",
    "message": "[ 错误信息 ]"
  }
}
```

²² 该部分将在错误返回中显示空值

5.1.1.3 哈希校验（任选项）

我们强烈建议运营商使用哈希校验以保护每个请求。若运营商选择使用哈希校验，PG SOFT 系统将对每个请求头字段的哈希信息进行验证。

注：

- 哈希校验默认处于关闭状态。如运营商需启用哈希校验，请联系 PG SOFT 提出申请。
- 运营商应根据收到的请求参数进行加密和计算，且主体参数不应硬编码。
- 哈希标头支持除 GetLaunchURLHTML 和 GetLaunchURLSDK 外的所有 API。

5.1.1.3.1 标头字段

下表描述了下述示例中的各种请求头：

构件	描述
Host	API 域名 例子： <code>apiexample.pgsoft.com</code>
x-date	以 YYYYMMDD 为格式的当前日期（UTC） 例子： <code>20190902</code>
x-content-sha256	请求主体内容字符串的 SHA256 哈希值 例子： <code>1700116101f424b9f6fc695b4dbaf2b7b0ee763ba1b3b53298e3069143ed46f1</code>
Authorization	多个组成部分的组合，用于请求认证 例子： <code>PWS-HMAC-SHA256</code>

```
Credential=20190902/OPERATOR_TOKENEXAMPLE/pws/v1,SignedHeaders=host;x-content-sha256;x-date,Signature=d78220cf06ae85f9d1db11dad9c3fd926799619eab3d28574aadb8cf328cd7aa
```

例子

```
Host:
apiexample.pgsoft.com
x-date: 20190902
x-content-sha256:
1700116101f424b9f6fc695b4dbaf2b7b0ee763ba1b3b53298e3069143ed46f1
Authorization: PWS-HMAC-SHA256
Credential=20190902/OPERATOR_TOKENEXAMPLE/pws/v1,SignedHeaders=host;x-content-sha256;x-date,Signature=d78220cf06ae85f9d1db11dad9c3fd926799619eab3d28574aadb8cf328cd7aa
```

请求头: x-content-sha256

计算请求主体内容字符串的 SHA256 哈希值:

例子

POST 主体:

```
operator_token=abc123&secret_key=a1b25cde5f3gh46ijkl&count=5000&bet_type=1&row_version=1346592723000
```

x-content-sha256:

```
1700116101f424b9f6fc695b4dbaf2b7b0ee763ba1b3b53298e3069143ed46f1
```

请求头: 验证

下表描述了下述示例中 Authorization 标头值的各个组成部分:

构件	描述
Credential	您的访问密钥 ID 和范围信息, 包括用于计算签名的日期、运营商令牌和服务。 该字符串具有以下形式: <pre>{x-date}/{operator_token}/pws/v1</pre> 例子: <pre>20190902/OPERATOR_TOKENEXAMPLE/pws/v1</pre>

SignedHeaders	以分号分隔的请求头列表，用于计算签名。该列表仅包含标头名称，并且标头名称必须为小写。 固定值： host;x-content-sha256;x-date
Signature	以 64 个小写的十六进制字符表示的 256-bit 签名。 例子： d78220cf06ae85f9d1db11dad9c3fd926799619eab3d28574aad b8cf328cd7aa

要计算签名，您首先需要包含 *{host}* *{x-content-sha256}* *{x-date}* 的签名字符串。接着，您将可以使用签名密钥计算出签名字符串的 HMAC-SHA256 哈希值：

```
hmac-sha256({salt}, {host}{x-content-sha256}{x-date})
```

计算主体内容字符串的 HMAC-SHA256 哈希值：

例子

```
Salt           : SALTEXAMPLE
Host           : apiexample.pgsoft.com
x-date         : 20190902
x-content-sha256 :
1700116101f424b9f6fc695b4dbaf2b7b0ee763ba1b3b53298e3069143ed46f1
```

签名：

```
d78220cf06ae85f9d1db11dad9c3fd926799619eab3d28574aad  
b8cf328cd7aa
```

5.1.2 令牌验证

PG SOFT 游戏系统将调用此运营商 API 以检查玩家登录游戏时所生成的玩家令牌。

注解:

- HTML Scheme²³ 的授权模式需要使用此 API
- 在此模式下系统将自动创建新玩家

请求参数

API URL : {OperatorAPIDomain}/VerifySession
HTTP Method : POST

URL 参数:

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符 (GUID) 注: 请把参数值设置为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
operator_token	String	是	运营商独有的身份识别
secret_key	String	是	PG SOFT 与运营商之间共享密码
operator_player_session	String	是	游戏启动时, 运营商系统将生成令牌 (HTML Scheme ²⁴)
game_id	Integer	否	游戏的唯一标识符
ip	String	否	玩家 IP 地址
custom_parameter	String	否	HTML Scheme 中 <i>extra_args</i> 参数的 <i>operator_param</i> 值

²³ 请参考 [HTML Scheme 方式](#) 的部分以获取登录流程信息

²⁴ 请参考 [HTML Scheme 方式](#) 的部分以获取更多信息

bet_type	Integer	是	游戏启动模式 ²⁵
----------	---------	---	----------------------

例子:

URL:

https://operator.api/VerifySession?trace_id=b3f37e57-2873-40b1-aa95-f126c25ed311

主体:

operator_token=abc123&secret_key=a1b25cde5f3gh46ijkl&operator_player_session=a2b3c4d5e6f7g8&ip=1.1.1.1&custom_parameter=operatorparam123&game_id=1&bet_type=1

验证方式

运营商需执行以下参数验证:

参数名称	描述	预期结果
operator_token	确保参数值与 PG SOFT 所提供的完全相同	不匹配时返回错误
secret_key		
operator_player_session	确保运营商玩家会话与 HTML Scheme ²⁶ 中所生成的令牌相同	

返回结果

主体参数:

JSON 对象

参数名称	参数类型	必需项	描述
player_name	String	是	玩家独有的身份识别 - 玩家名称不区分大小写 - 仅允许使用字母, 数字以及 "@"、"-","_" 符号 注: 上限 50 字符
nickname	String	否	玩家昵称 注: 上限 50 字符
currency	String	是	玩家选择的币种

²⁵ 请参考 [投注类型](#) 的部分以获取完整列表

²⁶ 请参考 [HTML Scheme 方式](#) 的部分以获取更多信息

例子:

```
{
  "data": {
    "player_name": "player1",
    "nickname": "nickname",
    "currency": "CNY"
  },
  "error": null
}
```

错误返回

代码	信息
1034	无效请求
1200	内部服务器错误

5.2 PG SOFT 提供对接 API

5.2.1 API 格式

5.2.1.1 请求参数

该 API 使用 HTTP 表单方式以及 RESTful 终点架构。PG SOFT 将根据以下内容格式发送请求：

`Content-Type: application/x-www-form-urlencoded`

5.2.1.2 返回结果

API 返回 JSON 格式的返回结果。对于成功和失败的 API 请求，运营商需返回 HTTP 状态码为 200 的 API 响应结果。

API 对于全部 API 方式都使用全球 JSON 格式的返回结果，其内容类型如下：

`Content-Type: application/json`

参数：

参数名称	参数类型	描述
data ²⁷	JSON Object	API 方式的返回结果信息
error ²⁸	JSON Object	发生错误或异常时的错误信息

²⁷该部分将在错误返回中显示空值

²⁸ 该部分将在成功返回 API 中显示空值。请参考[错误代码](#)的部分以获取错误返回的格式

例子:

成功返回

```
{  
  "data": {  
    [API 返回信息。JSON 格式将根据各个 API 而有所不同]  
  },  
  "error": null,  
}
```

失败/错误返回

```
{  
  "data": null,  
  "error": {  
    "code": "[ 错误代码]",  
    "message": "[ 错误信息]"  
  }  
}
```

5.2.2 查询玩家的钱包余额

查询特定玩家的钱包余额。

请求参数

API URL : {PgSoftAPIDomain}/Cash/v4/GetPlayerWallet

HTTP Method : POST

URL 参数:

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符 (GUID) 注: 请把参数值设置为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
operator_token	String	是	运营商独有的身份识别
secret_key	String	是	PG SOFT 与运营商之间共享密码
player_name	String	是	玩家独有的身份识别 玩家名称不区分大小写

例子:

URL:

https://api.pg-bo.me/external/Cash/v4/GetPlayerWallet?trace_id=b3f37e57-2873-40b1-aa95-f126c25ed311

主体:

operator_token=abc123&secret_key=a1b25cde5f3gh46ijkl&player_name=player123

返回结果

主体参数：

JSON 对象

参数名称	参数类型	描述
currencyCode	String	玩家选择的币种
totalBalance	Decimal	所有玩家钱包余额的总和： • 现金余额 • 红利余额 • 免费游戏余额
cashBalance	Decimal	玩家的现金钱包余额 运营商可参考该值作为玩家余额。
totalBonusBalance	Decimal	玩家的红利余额 这将显示玩家的所有可用红利余额，它无法被转出。
freeGameBalance	Decimal	玩家的免费游戏钱包余额 这将显示玩家的所有可用的免费游戏，它无法被转出。
bonuses	JSON Array	玩家的红利详情
freeGames	JSON Array	玩家的免费游戏详情
cashWallet	JSON Object	玩家的现金钱包详情
key	String	现金钱包的唯一标识符
cashId	Integer	玩家现金钱包的唯一标识符
cashBalance	Decimal	玩家的现金余额
bonusWallet	JSON Object	玩家的红利钱包详情
totalBonusBalance	Decimal	玩家的红利钱包余额总和
freeGameWallet	JSON Object	玩家的免费游戏钱包详情
freeGameBalance	Decimal	玩家的免费游戏钱包余额总和

JSON 数组 (红利游戏)

参数名称	参数类型	描述
key	String	红利钱包的唯一标识符
bonusId	Integer	红利钱包的唯一标识符 只有在红利游戏分配给玩家后才会出现。
bonusName	String	红利游戏名称 只有在红利游戏分配给玩家后才会出现。
transactionId	String	交易的唯一标识符
bonusParentType	String	母红利类型： 红利：正常创建的红利游戏 免费游戏：从免费游戏转换的红利游戏
gameIds	Integer Array	游戏的唯一标识符
gameIdLock	Integer	未完成的游戏 ID
balanceAmount	Decimal	玩家的红利余额
bonusRatioAmount	Decimal	红利游戏的流水要求 指红利游戏里的所赢金额在进入红利钱包之后，需要完成的总金额。
minimumConversionAmount	Decimal	红利游戏的最低现金转换金额
maximumConversionAmount	Decimal	红利游戏的最高现金转换金额
status	Integer	红利钱包的状态： 0：无效（已取消） 1：活跃 2：已过期 3：已转换 4：已完成 5：全新 6：已取消用户 7：已锁定

		8: 待处理
createdDate	Long	红利游戏的创建日期 (Unix 时间戳, 以毫秒为单位)
expiredDate	Long	红利游戏的截止日期 (Unix 时间戳, 以毫秒为单位)
createdBy	String	创建红利游戏的 API 或 BackOffice 用户
updatedBy	String	更新红利游戏的 API 或 BackOffice 用户
isSuppressDiscard	Boolean	允许玩家取消优惠: True: 不允许玩家取消优惠 False: 允许玩家取消优惠

JSON 数组 (免费游戏)

参数名称	参数类型	描述
key	String	免费游戏钱包的唯一标识符
freeGameId	Integer	免费游戏的唯一标识符 只有在免费游戏分配给玩家后才会出现。
freeGameName	String	免费游戏名称 只有在免费游戏分配给玩家后才会出现。
transactionId	String	交易的唯一标识符
gameIds	Integer Array	游戏的唯一标识符
gameIdLock	Integer	未完成的游戏 ID
gameCount	Integer	剩余的免费游戏总数
totalGame	Integer	免费游戏总数
balanceAmount	Decimal	玩家的免费游戏余额
minimumConversionAmount	Decimal	免费游戏的最低现金转换金额
maximumConversionAmount	Decimal	免费游戏的最高现金转换金额
multiplier	Integer	免费游戏的投注倍数
coinSize	Decimal	免费游戏的筹码大小

createdDate	Long	免费游戏的创建时间 (Unix 时间戳, 以毫秒为单位)
expiredDate	Long	免费游戏的截止时间 (Unix 时间戳, 以毫秒为单位)
status	Integer	免费游戏钱包的状态: 0: 无效 (已取消) 1: 活跃 2: 已过期 3: 已转换 4: 已完成 5: 全新 6: 已取消用户 7: 已封锁 8: 待处理
conversionType	Char	免费游戏完成后的转换类型: C: 现金 B: 红利
createdBy	String	在 API 或 BackOffice 创建免费游戏的用户
updatedBy	String	在 API 或 BackOffice 更新免费游戏的用户
isSuppressDiscard	Boolean	允许玩家取消优惠: True: 不允许玩家取消优惠 False: 允许玩家取消优惠

例子:

```
{
  "data": {
    "currencyCode": "CNY",
    "totalBalance": 50732.39,
    "cashBalance": 48853.39,
    "totalBonusBalance": 1815.00,
    "freeGameBalance": 64.00,
    "bonuses": [
      {
        "key": "11019-97090_B",
        "bonusId": 11019,
        "bonusName": "PG Bonus ",
        "transactionId": "",
        "bonusParentType": "Bonus",
        "gameIds": [
          94,
          69,
          71
        ],
        "gameIdLock": 0,
        "balanceAmount": 1000.00,
        "bonusRatioAmount": 1000.00,
        "minimumConversionAmount": 0.00,
        "maximumConversionAmount": 0.00,
        "status": 5,
        "createdDate": 1615952183000,
        "expiredDate": 1640966399000,
        "createdBy": "operator1",
        "updatedBy": "operator1",
        "isSuppressDiscard": true
      }
    ],
    "freeGames": [
      {
        "key": "575-16084_G",
        "freeGameId": 575,
        "freeGameName": "FREE GAME",
        "transactionId": "",
        "gameIds": [
          33
        ],
        "gameIdLock": 0,
        "gameCount": 0,
        "totalGame": 100,
        "balanceAmount": 0.00,
        "minimumConversionAmount": 0.00,
        "maximumConversionAmount": 0.00,
        "multiplier": 6,
        "coinSize": 0.1,
        "createdDate": 1544582329000,
        "expiredDate": 1544630399000,
        "status": 3,
        "conversionType": "C",
        "createdBy": "operator2",

```



```

        "updatedBy": "operator2",
        "isSuppressDiscard": false
    }
},
"cashWallet": {
    "key": "0_C",
    "cashId": 0,
    "cashBalance": 48853.39
},
"bonusWallet": {
    "totalBonusBalance": 1815.00,
    "bonuses": [
        {
            "key": "11019-97090_B",
            "bonusId": 11019,
            "bonusName": "PG Bonus ",
            "transactionId": "",
            "bonusParentType": "Bonus",
            "gameIds": [
                94,
                69,
                71
            ],
            "gameIdLock": 0,
            "balanceAmount": 1000.00,
            "bonusRatioAmount": 1000.00,
            "minimumConversionAmount": 0.00,
            "maximumConversionAmount": 0.00,
            "status": 5,
            "createdDate": 1615952183000,
            "expiredDate": 1640966399000,
            "createdBy": "operator3",
            "updatedBy": "operator3",
            "isSuppressDiscard": true
        }
    ]
},
"freeGameWallet": {
    "freeGameBalance": 64.00,
    "freeGames": [
        {
            "key": "575-16084_G",
            "freeGameId": 575,
            "freeGameName": "FREE GAME",
            "transactionId": "",
            "gameIds": [
                33
            ],
            "gameIdLock": 0,
            "gameCount": 0,
            "totalGame": 100,
            "balanceAmount": 0.00,
            "minimumConversionAmount": 0.00,
            "maximumConversionAmount": 0.00,
            "multiplier": 6,
            "coinSize": 0.1,

```

```

    "createdDate": 1544582329000,
    "expiredDate": 1544630399000,
    "status": 3,
    "conversionType": "B",
    "createdBy": "operator4",
    "updatedBy": "operator4",
    "isSuppressDiscard": false
  }
]
},
"error": null
}

```

错误返回

代码	信息
1305	无效玩家 – 玩家不存在: [玩家名称]

5.2.3 充值与转出流程

PG SOFT 系统正在为所有 API 操作实现**幂等操作**。根据以下情况，要求运营商通过使用相同的 `transfer_reference` **重新发送** 请求：

- 从 PG SOFT 系统中收到错误的返回（错误的返回格式）。
- 收到除了 3001、3005、3100 的错误返回代码。
- 没有收到 PG SOFT 系统的返回。

*** 注：**`transfer_reference` 是独特的，且每个转账请求仅限一个。请避免在充值或转出流程中使用相同的 `transfer_reference`。

实际转账金额

为防止因货币基准汇率换算错误导致的损失，PG SOFT 对转账钱包 API 进行了更新，新增参数 `real_transfer_amount/realTransferAmount`，用于 API 的请求和响应中。

`real_transfer_amount` 表示请求中金额的实际数值。运营商必须在请求中包含该参数，并验证 API 响应中返回的 `realTransferAmount` 是否与原始值匹配。

若校验过程中发现不一致，运营商不得更新玩家的钱包余额，且必须向 PG SOFT 返回错误响应。

例子：

若某货币的基础单位为 1000，运营商采用 1:1000 的换算比例，且请求中的 `transfer_amount` 为 1，则 `real_transfer_amount` 应为 1000。

若 PG SOFT 收到的 `real_transfer_amount` 不正确，API 将返回错误响应给运营商。

5.2.3.1 充值

充值现金至特定玩家的现金钱包。

注解:

- Transfer_reference 是每笔交易的唯一识别码。运营商需要为每笔传送请求发送新的 transfer_reference。
- 对于重复请求，PG SOFT 不对玩家钱包余额进行任何操作，但返回转账成功的响应。

请求参数

API URL : {PgSoftAPIDomain}/Cash/v4/TransferIn
HTTP Method : POST

URL 参数:

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符 (GUID) 注: 请把参数值设置为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
operator_token	String	是	运营商独有的身份识别
secret_key	String	是	PG SOFT 与运营商之间共享密码
player_name	String	是	玩家独有的身份识别 玩家名称不区分大小写
amount	Decimal	是	转账金额 (小数点后两位) 注: 最低 0.01
transfer_reference	String	是	交易凭证, 运营商平台生成并确保每次交易是独有的。 注: 上限 50 字符
currency	String	是	玩家选择的币种
real_transfer_amount	Decimal	是	真实世界中交易金额的实际数值

注：PG SOFT 将对运营商发送的数值进行校验

例子：

URL:

https://api.pg-bo.me/external/Cash/v4/TransferIn?trace_id=b3f37e57-2873-40b1-aa95-f126c25ed311

主体:

operator_token=abc123&secret_key=a1b25cde5f3gh46ijkl&player_name=player123&amount=1&transfer_reference=ref1530879795000¤cy=IDR&real_transfer_amount=1000

返回结果

主体参数：

JSON 对象

参数名称	参数类型	描述
transactionId	String	特殊交易识别码
balanceAmountBefore	Decimal	交易前的玩家余额
balanceAmount	Decimal	交易后的玩家余额
amount	Decimal	交易金额
realTransferAmount	Decimal	真实世界中交易金额的实际数值 注： 若未满足以下任一条件，运营商需对该数值进行校验并通知 PG SOFT： - 对于基础单位为 1:1 的货币： $realTransferAmount = Amount$ - 对于基础单位为 1:1000 且由 PG SOFT 处理换算的货币： $realTransferAmount = Amount$ - 对于基础单位为 1:1000 且由运营商处理换算（默认）的货币： $realTransferAmount = Amount * 1000$
transactionType	Integer	交易的标识符

		交易类型： 100: TransferInCash 200: TrasnferOutCash 209: TransferAllOutCash
--	--	--

例子：

```
{
  "data": {
    "transactionId": 35509540,
    "balanceAmountBefore": 1.00,
    "balanceAmount": 2.00,
    "amount": 1.00
    "realTransferAmount": 1000.00
    "transactionType": 100
  },
  "error": null
}
```

错误返回

代码	信息
1034	请求无效
1204	获取运营商失败
1305	无效玩家 – 玩家不存在：[玩家名称]
1315	玩家操作进行中
3001	不能空值
3005	玩家钱包不存在
3052	服务异常：货币代码不匹配
3071	服务异常：交易凭证 ID 已被使用，且不属于该玩家
3100	转账失败
3101	转账请求中，请重试查看最新状态
3107	配置无效

5.2.3.2 转出余额

将特定玩家的现金钱包的余额转出。

注解:

- Transfer_reference 是每笔交易的唯一识别码。运营商需要为每笔传送请求发送新的 transfer_reference。
- 对于重复请求，PG SOFT 不对玩家钱包余额进行任何操作，但返回转账成功的响应。

请求参数

API URL : {PgSoftAPIDomain}/Cash/v4/TransferOut

HTTP Method : POST

URL 参数:

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符 (GUID) 注: 请把参数值设置为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
operator_token	String	是	运营商独有的身份识别
secret_key	String	是	PG SOFT 与运营商之间共享密码
player_name	String	是	玩家独有的身份识别 玩家名称不区分大小写
amount	Decimal	是	转账金额 (小数点后两位) 注: 最低 0.01
transfer_reference	String	是	交易凭证, 运营商平台生成并确保每次交易是独有的。 注: 上限 50 字符
currency	String	是	玩家选择的币种

real_transfer_amount	Decimal	是	真实世界中交易金额的实际数 注：PG SOFT 将对运营商发送的数值进行校验
-----------------------------	---------	---	--

例子：

URL:

https://api.pg-bo.me/external/Cash/v4/TransferOut?trace_id=b3f37e57-2873-40b1-aa95-f126c25ed311

主体:

operator_token=abc123&secret_key=a1b25cde5f3gh46ijkl&player_name=player123&amount=1&transfer_reference=ref1530879795000¤cy=IDR&real_transfer_amount=1000

返回结果

主体参数：

JSON 对象

参数名称	参数类型	描述
transactionId	String	特殊交易识别码
balanceAmountBefore	Decimal	交易前的玩家余额
balanceAmount	Decimal	交易后的玩家余额
amount	Decimal	交易金额
realTransferAmount	Decimal	真实世界中交易金额的实际数值 注： 若未满足以下任一条件，运营商需对该数值进行校验并通知 PG SOFT： - 对于基础单位为 1:1 的货币： $\text{realTransferAmount} = \text{Amount}$ - 对于基础单位为 1:1000 且由 PG SOFT 处理换算的货币： $\text{realTransferAmount} = \text{Amount}$ - 对于基础单位为 1:1000 且由运营商处理换算（默认）的货币： $\text{realTransferAmount} = \text{Amount} * 1000$
transactionType	Integer	交易的标识符

	交易类型
	100: TransferInCash
	200: TrasnferOutCash
	209: TransferAllOutCash

例子:

```
{
  "data": {
    "transactionId": 35509540,
    "balanceAmountBefore": 2.00,
    "balanceAmount": 1.00,
    "amount": 1.00
    "realTransferAmount": 1000.00
    "transactionType": 200
  },
  "error": null
}
```

错误返回

代码	信息
1034	请求无效
1204	获取运营商失败
1305	无效玩家 – 玩家不存在: [玩家名称]
1315	玩家操作进行中
3001	不能空值
3005	玩家钱包不存在
3013	服务异常: 余额不足无法转出
3052	服务异常: 货币代码不匹配
3071	服务异常: 交易凭证 ID 已被使用, 且不属于该玩家
3100	转账失败
3101	转账请求中, 请重试查看最新状态
3107	配置无效

5.2.4 钱包操作限制及处理

为了避免玩家余额报错，同时进行钱包转账和启动游戏是被禁止的。在这个情况，PG SOFT 将限制玩家操作并会对该钱包 API 请求返回错误代码：

```
1315: Player's operation in progress
```

错误处理

3004

若在创建玩家时发生无法处理的错误，PG SOFT 将返回此错误代码。运营商可通过 External API 重新创建玩家钱包或重新启动 PG SOFT 游戏以自动创建玩家。

1315

当遇到以下情况时，PG SOFT 将返回此错误代码：

- 并发动作（例如：该玩家同时使用两个不同的浏览器启动游戏等）。
- 玩家在进行交易的同时启动游戏或在游戏中进行旋转。
- 玩家钱包不存在。
- 在进行钱包转账时出现错误。

若运营商收到此错误代码，应在 5 分钟后重新发送钱包交易请求，并需要使用相同的 transfer_reference 重新发送请求。

5.2.5 转出所有余额（任选项）

从特定玩家的现金钱包中转出所有钱包金额。

注解：

- Transfer_reference 是每笔交易的唯一识别码。运营商需要为每笔传送请求发送新的 transfer_reference。
- 对于重复请求，PG SOFT 不对玩家钱包余额进行任何操作，但返回转账成功的响应。
- 此 API 用于清空玩家的钱包余额。如若玩家没有任何余额时，系统将不会显示任何错误返回。运营商需要在 API 响应中检查玩家之前和当前的剩余金额，并且确认实际提款金额以避免余额丢失。

请求参数

API URL : {PgSoftAPIDomain}/Cash/v4/TransferAllOut

HTTP Method : POST

URL 参数：

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符（GUID） 注：请把参数值设置为 GUID 格式

主体参数：

参数名称	参数类型	必需项	描述
operator_token	String	是	运营商独有的身份识别
secret_key	String	是	PG SOFT 与运营商之间共享密码
player_name	String	是	玩家独有的身份识别 • 玩家名称不区分大小写
transfer_reference	String	是	交易凭证，运营商平台生成并确保每次交易是独有的。 注：上限 50 字符
currency	String	否	玩家选择的币种

例子:

URL:

https://api.pg-bo.me/external/Cash/v4/TransferAllOut?trace_id=b3f37e57-2873-40b1-aa95-f126c25ed311

主体:

operator_token=abc123&secret_key=a1b25cde5f3gh46ijkl&player_name=player123&transfer_reference=ref1530879795000¤cy=IDR

返回结果

主体参数:

JSON 对象

参数名称	参数类型	描述
transactionId	String	特殊交易识别码
balanceAmountBefore	Decimal	交易前的玩家余额
balanceAmount	Decimal	交易后的玩家余额
amount	Decimal	交易金额
realTransferAmount	Decimal	真实世界中交易金额的实际数值 注: 若未满足以下任一条件, 运营商需对该数值进行校验并通知 PG SOFT: - 对于基础单位为 1:1 的货币: $\text{realTransferAmount} = \text{Amount}$ - 对于基础单位为 1:1000 且由 PG SOFT 处理换算的货币: $\text{realTransferAmount} = \text{Amount}$ - 对于基础单位为 1:1000 且由运营商处理换算 (默认) 的货币: $\text{realTransferAmount} = \text{Amount} * 1000$
transactionType	Integer	交易的标识符 交易类型: 100: TransferInCash 200: TransferOutCash

209: TransferAllOutCash

例子:

```
{
  "data": {
    "transactionId": 12458848,
    "balanceAmountBefore": 1.00,
    "balanceAmount": 0,
    "amount": 1.00
    "realTransferAmount": 1000.00
    "transactionType": 209
  },
  "error": null
}
```

错误返回

代码	信息
1034	请求无效
1204	获取运营商失败
1305	无效玩家 – 玩家不存在: [玩家名称]
1315	玩家操作进行中
3001	不能空值
3005	玩家钱包不存在
3071	服务异常: 交易凭证 ID 已被使用, 且不属于该玩家
3107	配置无效

5.2.6 获取单个交易记录（任选项）

透过搜索交易凭证及玩家账号，可获取特定玩家的单个交易记录。

注解：

- 此 API 只能在最后一笔交易的 5 分钟后调用。
- 请避免在转账处理过程中使用此 API 查询交易状态，否则因处理延迟而导致查询结果不准确。
- 此 API 数据仅保留最多 180 天。

请求参数

API URL : {PgSoftAPIDomain}/Cash/v4/GetSingleTransaction
HTTP Method : POST

URL 参数：

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符（GUID） 注：请把参数值设置为 GUID 格式

主体参数：

参数名称	参数类型	必需项	描述
operator_token	String	是	运营商独有的身份识别
secret_key	String	是	PG SOFT 与运营商之间共享密码
transfer_reference	String	是	交易凭证，运营商平台生成并确保每次交易是独有的。 注：上限 50 字符

例子：

URL:

https://api.pg-bo.me/external/Cash/v4/GetSingleTransaction?trace_id=b3f37e57-2873-40b1-aa95-f126c25ed311

主体:

operator_token=abc123&secret_key=a1b25cde5f3gh46ijkl&transfer_reference=ref1530879795000

返回结果

主体参数:

JSON 对象

参数名称	参数类型	描述
transactionId	String	特殊交易识别码
playerName	String	玩家账号
currencyCode	String	玩家选择的币种
transactionType	Integer	交易类型: 100: TransferInCash 200: TransferOutCash
transactionAmount	Decimal	交易金额
transactionFrom	Decimal	交易前的钱包余额
transactionTo	Decimal	交易后的钱包余额
transactionDateTime	Long	交易的日期和时间 (Unix 时间戳, 以毫秒为单位)

例子:

```
{
  "data": {
    "transactionId": 35595943,
    "playerName": "player123",
    "currencyCode": "EUR",
    "transactionType": 100,
    "transactionAmount": 1000000.00,
    "transactionFrom": 0.00,
    "transactionTo": 1000000.00,
    "transactionDateTime": 1522984865000
  },
  "error": null
}
```

错误返回

代码	信息
3040	交易不存在

5.2.7 投注详情页面（任选项）

显示玩家的投注结果和特定投注的详细信息。

步骤 1

在获取游戏记录详细图片之前，必须先获取运营商令牌。（限定时间：30 分钟）。

请求参数

API URL : {PgSoftAPIDomain}/Login/v1/LoginProxy

HTTP Method : POST

URL 参数:

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符 (GUID) 注: 请把参数值设置为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
operator_token	String	是	运营商独有的身份识别
secret_key	String	是	PG SOFT 与运营商之间共享密码

例子:

URL:

https://api.pg-bo.me/external/Login/v1/LoginProxy?trace_id=b3f37e57-2873-40b1-aa95-f126c25ed311

主体:

operator_token=abc123&secret_key=a1b25cde5f3gh46ijk1

返回结果

主体参数:

JSON 对象

参数名称	参数类型	描述
operator_session	String	运营商令牌

例子:

```
{
  "data": {
    "operator_session": E4ECB5CF-0BC6-4DA8-8551-8526F48AB9CD
  },
  "error": null
}
```

步骤 2

通过以下方式获取投注详情:

API URL : {PgSoftAPIDomain}/external-game-launcher/api/v1/GetLaunchURLHTML

HTTP Method : POST

URL 参数:

参数名称	参数类型	描述
trace_id	String	请求的唯一标识符 (GUID) 注: 请把参数值设置为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
path	varchar (500)	是	投注详情页面路径: /history/redirect.html
extra_args	Query String Paramater	是	投注详情页面参数 (请参考相关参数列表) 注: - 请使用 <code>UrlEncode</code> 对值进行编码, 以避免未知错误 - 最多 500 字符

url_type	varchar (50)	是	URL 类型: web-history: 投注详情页面
client_ip	varchar (50)	是	玩家 IP
group_id	UInt16	否	运营商分组 ID

extra_args 参数:

参数名称	参数类型	描述
t	String	运营商令牌
psid	String	母注单 ID
sid	String	投注 ID
gid	String	游戏 ID
lang	String	语言: • 英文 (默认) • 中文
type	String	固定值: operator

例子:

主体:

```
path=/history/redirect.html&extra_args=trace_id%3Dde335d4d-b133-4422-9c6e-8554cb1fbc7e%26t%3DE4ECB5CF-0BC6-4DA8-8551-8526F48AB9CD%26psid%3D1821471534863220736%26sid%3D1821471534863220736%26lang%3Den%26type%3Doperator&url_type=web-history&client_ip=1.1.1.1
```

返回结果

HTTP 响应主体:

浏览器只需执行 HTML 页面即可启动游戏。

***注: 返回的 HTML 代码内容会根据实际情况进行调整及优化, 并无固定格式。请勿通过反序列化或解析方式取得最终的游戏启动 URL。**

返回的 HTTP 状态码：

状态码	描述	原因	解决办法
200	成功返回	-	-
400	无效参数	参数未按照要求提供	检查 HTTP 方法、HTTP 标头和 HTTP 主体是否按照要求进行请求。
404	路径未找到	无效路径	检查路径是否正确
500	内部服务器错误	发生内部服务器错误	• 重试 • 通知 PG SOFT
504	网关超时	请求未能成功到达目标服务器，且某个代理服务器发生了故障	• 重试 • 通知 PG SOFT

HTML 响应例子（仅供参考）：

```
<!doctype html><html lang="en">...</html>
```

5.2.8 获取多个玩家钱包余额（任选项）

获取多个玩家的钱包余额（最多 10 个玩家）。

注解:

- 最多可获取 10 个玩家的钱包余额。

请求参数

API URL : {PgSoftAPIDomain}/Cash/v4/GetPlayersWallet
HTTP Method : POST

URL 参数:

参数名称	参数类型	必需项	描述
trace_id	String	是	请求的唯一标识符 (GUID) 注: 请把参数值设置为 GUID 格式

主体参数:

参数名称	参数类型	必需项	描述
operator_token	String	是	运营商独有的身份识别
secret_key	String	是	PG SOFT 与运营商之间共享密码
player_names	String Array	是	玩家独有的身份识别 玩家名称不区分大小写

例子:

URL:

https://api.pg-bo.me/external/Cash/v4/GetPlayersWallet?trace_id=b3f37e57-2873-40b1-aa95-f126c25ed311

主体:

operator_token=abc123&secret_key=a1b25cde5f3gh46ijkl&player_names=player1&player_names=player2

返回结果

主体参数：

JSON 对象

参数名称	参数类型	描述
totalCount	Integer	记录总数
totalPage	Integer	可用批次的总数
result	JSON Array	玩家钱包详情列表

JSON 数组

参数名称	参数类型	描述
playerName	String	玩家名称
currencyCode	String	玩家选择的币种
totalBalance	Decimal	玩家钱包余额 注： • 显示的余额于最近更新时间，并非实时余额 • 此处仅显示现金余额，不包含没费游戏余额或红利余额 • 若需获取实时余额，请使用【5.2.2 查询玩家的钱包余额】
updatedAt	Long	玩家钱包最近更新时间 (Unix 时间戳，以毫秒为单位)

例子:

```
{
  "data": {
    "totalCount": 2,
    "totalPage": 1,
    "result": [
      {
        "updatedAt": 1729501647077,
        "playerName": "player1",
        "currencyCode": "CNY",
        "totalBalance": 33.59
      },
      {
        "updatedAt": 1724062855673,
        "playerName": "player2",
        "currencyCode": "CNY",
        "totalBalance": 2000061103.84
      }
    ]
  },
  "error": null
}
```

错误返回

代码	信息
3001	不能空值

5.2.9 其他（任选项）

PG SOFT 为不同的模块提供了几个 API，可依据选择对接。运营商可根据需求进行对接：

- **投注历史记录** : 获取玩家历史记录。
- **游戏列表** : 获取最新游戏列表和可用的投注大小。
- **免费游戏** : 通过 API 执行多项操作（例如：创建免费游戏，给予玩家免费游戏，及取消免费游戏等）。
- **红利游戏** : 通过 API 执行多项操作（例如：创建红利游戏，给予玩家红利，及取消红利等）。
- **锦标赛** : 通过 API 将玩家添加到锦标赛中，并且获得锦标赛排名。

5.3 PG SOFT 游戏对接运营商 iOS 应用程序

运营商的应用程序需满足以下条件，才可在应用程序客户端中打开 PG SOFT 游戏

- iOS 8.0 或以上
- WKWebView

在设备未达到最低配置要求或是网络情况未达要求时，PG SOFT 游戏将出现“体验欠佳”的提示。玩家仍可继续进入游戏，但无法保证可获得最佳体验及稳定的 PG SOFT 游戏。

为保证应用程序客户端能通过 PG SOFT 游戏要求的检测，运营商需在初始化 WKWebView 组件时，添加以下代码：

```
WKUserContentController *controller = [[WKUserContentController alloc] init];  
[controller addScriptMessageHandler:self name:@"Could be any string value"];  
  
WKWebViewConfiguration *configuration = [[WKWebViewConfiguration alloc] init];  
configuration.userContentController = controller;  
  
WKWebView *webView = [[WKWebView alloc] initWithFrame: CGRectZero  
configuration:  
configuration];  
webView.uiDelegate = self;  
self.view = webView;
```

此操作是为了构造 WKUserContentController 对象，并在配置上设置其 userContentController 属性。请注意 userContentController 必须至少有一个 scriptMessageHandler（通过 addScriptMessageHandler 分配）。

更多关于 WKWebView 的详细说明，请参考 [Apple Developer Documentation](#)

6 解决待处理投注

在某些情况下，玩家于游戏期间可能发生不可预见的情况，如网络延误、系统故障、系统超时等等。这将可能导致待处理投注的出现。

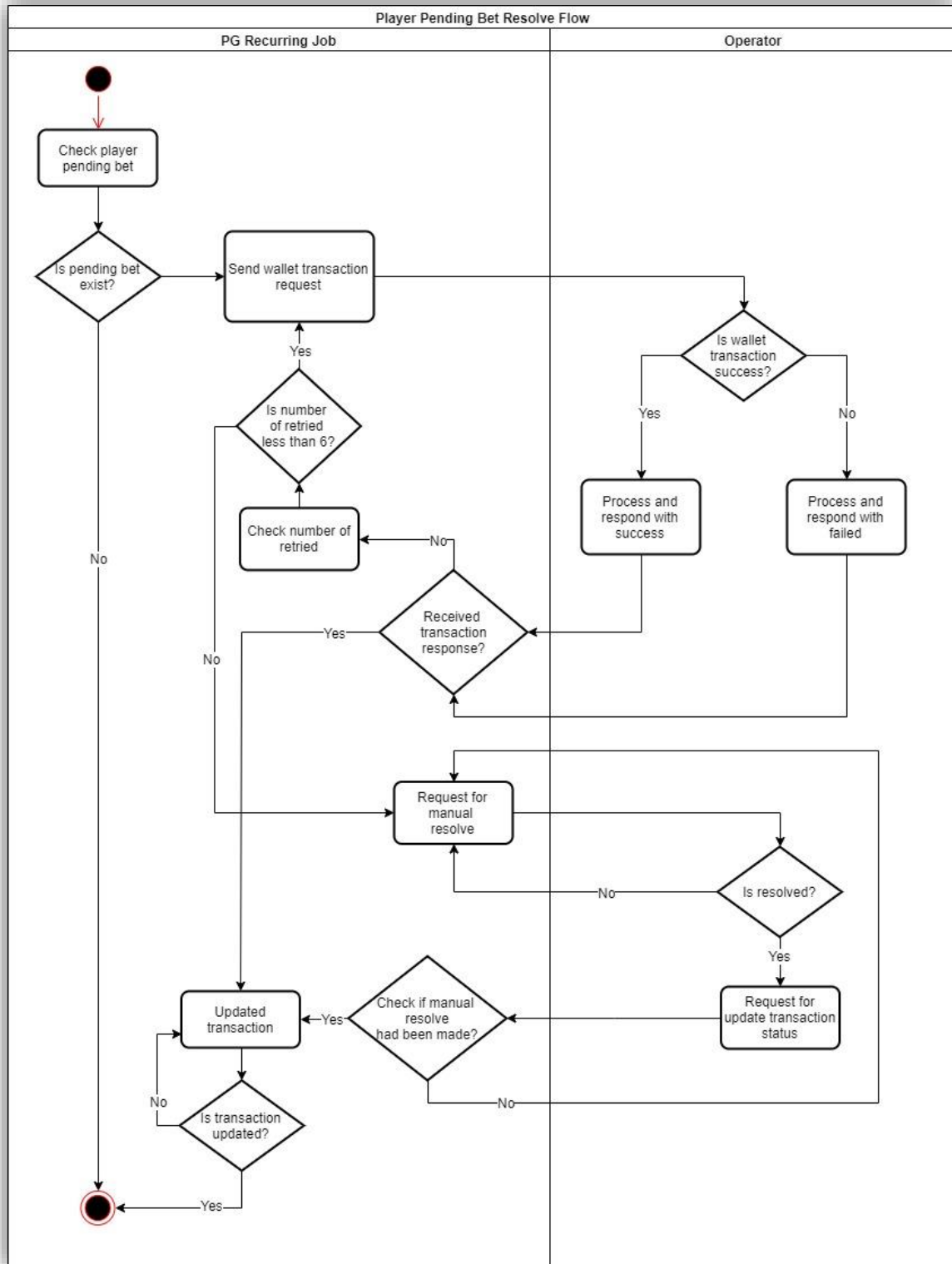
通常，当玩家重新启动游戏时，系统将自动解决该待处理投注。但是在某些罕见的情况下，投注将无法获得解决；并保持待处理状态：

- 当玩家遇到错误后，没有重新启动游戏；
- 解决待处理投注时，PG SOFT 系统未收到来自运营商的任何响应；
- 解决待处理投注时，PG SOFT 系统会收到来自运营商的错误响应。

解决待处理投注的默认间隔时间设置为 60 分钟。超过间隔的待处理投注将由 PG SOFT 系统自动解决。该间隔是可自定义的，每个运营商都可自定义解决待处理投注的间隔时间。

若尝试几次后仍无法解决待处理投注，则需要手动解决。在这种情况下，PG SOFT 将要求运营商手动解决待处理投注。在获得运营商确认后，PG SOFT 将会手动把该投注的状态标记为完成。

截图



玩家待处理投注的解决流程

7 重置游戏状态

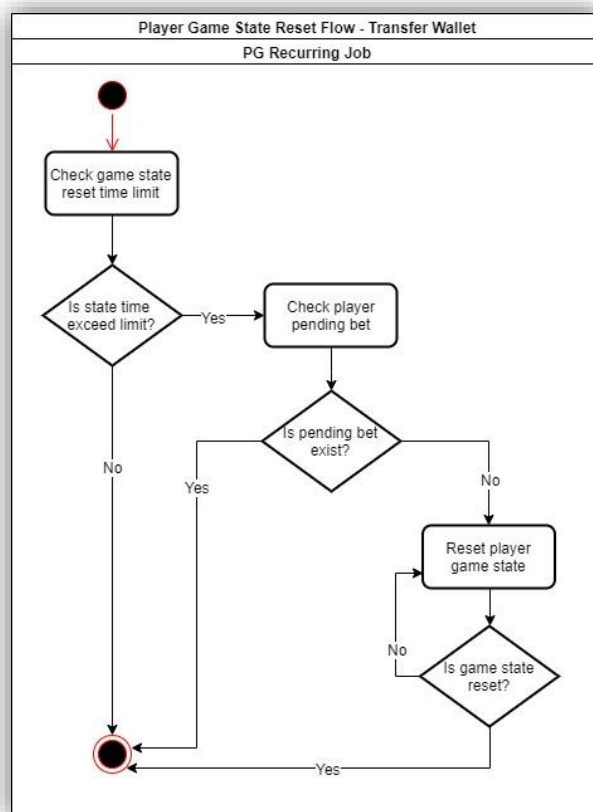
所有游戏都支持恢复游戏状态的功能，该功能允许玩家在任何设备回到上一个游戏状态并继续游戏。举例，玩家在赢得免费旋转后关闭了游戏，但玩家依然可以在几天后继续进行免费旋转。

但是，默认情况下，每 90 天重置玩家的游戏状态。此间隔是可自定义的，每个运营商都可自定义游戏状态重置的间隔时间。系统将清除所有已完成和未完成的游戏进度，并且玩家在重置游戏后将无法继续游戏。

对于转账钱包模式，默认情况下每 90 天将重置每个玩家的游戏状态：

- 若无任何待处理投注，PG SOFT 系统将自动重置玩家的游戏状态。

截图



转账钱包的玩家游戏状态重置流程

8 限制

8.1 IP 限制

来自以下限制区域的玩家将无法访问 PG SOFT 游戏：

- 马来西亚
- 新加坡
- 台湾
- 美国
- 澳门
- 以色列
- 伊朗
- 北韩
- 澳大利亚
- 阿拉伯联合酋长国
- 英国

8.2 货币限制

以下为不被 PG SOFT 所支持的货币：

- AUD
- BGN
- CSD
- ESP
- HRK
- STD
- TRL
- HKD
- GBP
- JYP (不接受新请求)
- MOP
- MYR
- SGD
- TWD
- 除货币代码附录所列之外的所有加密货币

9 附录

9.1 错误代码

错误代码	描述
1034	无效请求
1035	行动失败
1200	内部服务器失败
1204	获取运营商失败
1300	无效玩家令牌
1301	玩家令牌空置
1302	无效玩家令牌
1303	发生服务器错误
1305	无效玩家 – 玩家不存在: [玩家名称]
1306	玩家被阻止访问当前游戏
1307	无效玩家令牌
1308	玩家令牌已过期
1309	玩家已被封锁
1310	运营商和玩家令牌验证失败
1315	玩家操作进行中
1400	游戏正在维护中
1401	游戏无效
1402	游戏不存在
3001	不能空值
3004	玩家不存在
3005	玩家钱包不存在
3006	玩家钱包已存在
3009	免费游戏不存在
3013	服务异常: 余额不足无法转出

3014	免费游戏无法取消
3019	免费游戏不足
3021	投注不存在
3022	投注已支付
3030	免费游戏过期
3031	免费游戏已经转换
3032	投注已存在
3033	投注失败
3034	支付失败
3035	倍数错误
3036	余额不足无法转换
3040	交易不存在
3202	余额不足无法投注

9.2 货币代码

货币代码	货币符号	货币名称	基础单位
AED	د.إ.	阿拉伯联合酋长国迪拉姆	1
AFN	؍	阿富汗	1
ALL	L	阿尔巴尼亚列克	1
AMD	֏	亚美尼亚德拉姆	1
ANG	ƒ	安的列斯盾	1
AOA	Kz	安哥拉宽扎	1
ARS	\$	阿根廷比索	1
ARSB	\$	阿根廷比索蓝市美元	1
AWG	ƒ	阿鲁巴弗罗林	1
AZN	₼	阿塞拜疆马纳特	1
BAM	KM	波黑可兑换马克	1
BBD	\$	巴巴多斯元	1
BDT	Tk	孟加拉塔卡	1
BHD	BD	巴林第纳尔	1
BIF	FBu	布隆迪法郎	1000
BMD	\$	百慕大元	1
BND	B\$	汶莱元	1
BOB	\$b	玻利维亚诺	1
BRL	R\$	巴西里亚伊（雷亚尔）	1
BSD	\$	巴哈马元	1
BTN	Nu.	不丹努尔特鲁姆	1
BWP	P	博茨瓦纳普拉	1
BYN	BYN	白俄罗斯卢布	1
BYR	Br	白俄罗斯卢布	1
BZD	BZ\$	伯利兹元	1
CAD	C\$	加拿大元	1

CDF	FC	刚果法郎	1000
CHF	Fr.	瑞士法郎	1
CLP	\$	智利比索	1
CNY	¥	人民币	1
COP	\$	哥伦比亚比索	1000
CRC	₡	哥斯达黎加科朗	1
CUP	₱	古巴比索	1
CVE	\$	佛得角埃斯库多	1
CZK	Kč	捷克克朗	1
DJF	Fdj	吉布提法郎	1
DKK	kr.	丹麦克朗	1
DOP	RD\$	多米尼加比索	1
DZD	DA	阿尔及利亚第纳尔	1
EGP	£	埃及镑	1
ERN	Nkf	厄立特里亚纳克法	1
ETB	ብር	埃塞俄比亚比尔	1
EUR	€	欧元	1
FJD	\$	斐济元	1
FKP	£	福克兰岛磅	1
GEL	ლ	格鲁吉亚拉里	1
GHS	GH¢	加纳塞地	1
GIP	£	直布罗陀庞德	1
GMD	D	冈比亚货币	1
GNF	FG	几内亚法郎	1000
GTQ	Q	危地马拉格查尔	1
GYD	\$	圭亚那元	1
HNL	L	洪都拉斯伦皮拉	1
HTG	G	海地古德	1
HUF	Ft	匈牙利福林	1

IDR	Rp	印度尼西亚卢比盾	1000
ILS	₪	以色列谢克尔	1
INR	₹	印度卢比	1
IQD	د.ع	伊拉克第纳尔	1000
IRR	ریال	伊朗里亚尔	1000
ISK	kr	冰岛克朗	1
JMD	J\$	牙买加元	1
JOD	د.أ	约旦第纳尔	1
KES	KSh	肯尼亚先令	1
KGS	KGS	吉尔吉斯斯坦索姆	1
KHR	៛	柬埔寨利尔斯	1000
KMF	CF	科摩罗法郎	1
KPW	₩	北朝鲜元	1
KRW	₩	韩元	1000
KWD	ك	科威特第纳尔	1
KYD	\$	开曼岛元	1
KZT	₸	坚戈	1
LAK	₭	老挝基普	1000
LBP	ل.ل	黎巴嫩镑	1000
LKR	Rs	斯里兰卡卢比	1
LRD	\$	黎巴嫩元	1
LSL	L	莱索托洛蒂	1
LVL	Ls	拉脱维亚拉特	1
LYD	LD	利比亚第纳尔	1
MAD	DH	摩洛哥迪拉姆	1
MBTC	mB	比特币（虚拟货币）	1
MDL	L	摩尔多瓦列伊	1
MGA	Ar	马达加斯加阿里亚里	1000
MKD	ден	马其顿第纳尔	1

MMK	K	缅甸元	1000
MNT	₮	蒙古圖格裡克	1000
MUR	Rs	毛里求斯卢比	1
MVR	Rf.	马尔代夫罗非亚	1
MWK	MK	马拉维克瓦查	1
MXN	\$	墨西哥比索	1
MZN	MT	莫桑比克梅蒂卡尔	1
NAD	\$	纳米比亚元	1
NGN	₦	尼日利亚奈拉	1
NIO	C\$	尼加拉瓜科多巴	1
NOK	kr	挪威克朗	1
NPR	Rs	尼泊尔卢比	1
NZD	\$	新西兰元	1
OMR	ريال	阿曼里亚尔	1
PAB	B/.	巴拿马巴波亚	1
PEN	S/	秘鲁索尔	1
PGK	K	巴布亚新几内亚基纳	1
PHP	₱	菲律宾披索	1
PKR	Rs	巴基斯坦卢比	1
PLN	zł	波兰兹罗提	1
PYG	Gs	巴拉圭瓜拉尼	1000
QAR	QR	卡塔尔里亚尔	1
RON	lei	罗马尼亚列伊	1
RSD	PCД	塞尔维亚第纳尔	1
RUB	₽	俄罗斯卢布	1
RWF	FRw	卢旺达法郎	1000
SAR	SR	沙特里亚尔	1
SBD	\$	所罗门群岛元	1
SCR	SR	塞舌尔卢比	1

SDG	ج.س.	苏丹镑	1
SEK	kr	瑞典克朗	1
SHP	£	圣赫勒拿磅	1
SLE	Le	塞拉利昂利昂	1
SLL	Le	塞拉利昂利昂	1000
SOS	S	索马里先令	1
SRD	\$	苏里南元	1
SVC	\$	萨尔瓦多科朗	1
SYP	£	叙利亚镑	1
SZL	L	斯威士马兰吉尼	1
THB	฿	泰铢	1
TJS	TJS	塔吉克斯坦索莫尼	1
TMT	T	土库曼斯坦新马纳特	1
TND	د.ت	突尼斯第纳尔	1
TOP	T\$	汤加潘加	1
TRX	TRX	波场加密货币	1
TRY	₺	土耳其里拉	1
TTD	TT\$	特立尼达与多巴哥元	1
TUSD	TUSD	TrueUSD (虚拟货币)	1
TZS	TSh	坦桑尼亚先令	1000
UAH	₴	乌克兰赫夫米	1
UBTC	μB	微比特币	1
UGX	USh	乌干达先令	1000
USD	\$	美元	1
USDC	USDC	USD Coin (虚拟货币)	1
USDT	₮	泰达币	1
UYU	\$U	乌拉圭比索	1
UZS	so'm	乌兹别克斯坦苏姆	1000
VES	Bs.S	委内瑞拉玻利瓦尔	1

VND	₫	越南盾	1000
VUV	VT	瓦努阿图瓦图	1
WST	\$	萨摩亚塔拉	1
XAF	FCFA	中非金融合作法郎	1
XCD	\$	东加勒比元	1
XOF	CFA	CFA 法郎	1
XPF	₣	太平洋法郎	1
YER	ريال	也门里亚尔	1
ZAR	R	南非兰特	1
ZMW	ZK	赞比亚克瓦查	1
GC	GC	Stake Gold Coins (社交娱乐场币)	1
SC	SC	Stake Cash (社交娱乐场币)	1

9.3 货币基础单位及处理

运营商在提交环境设置请求前，应始终核实货币列表中的基础单位。以下为基础单位为 1000 的货币列表。运营商需在集成应用中选择由 PG SOFT 或运营商自行负责该货币的转换处理。

- BIF (布隆迪法郎)
- CDF (刚果法郎)
- COP (哥伦比亚比索)
- GNF (几内亚法郎)
- IDR (印度尼西亚盾)
- IQD (伊拉克第纳尔)
- IRR (伊朗里亚尔)
- KHR (柬埔寨瑞尔)
- KRW (韩国韩元)
- LAK (老挝基普)
- LBP (黎巴嫩镑)
- MGA (马达加斯加阿里亚里)
- MMK (缅甸元)
- MNT (蒙古图格里克)
- PYG (巴拉圭瓜拉尼)
- RWF (卢旺达法郎)
- SLL (塞拉利昂利昂)
- TZS (坦桑尼亚先令)
- UGX (乌干达先令)
- UZS (乌兹别克斯坦苏姆)
- VND (越南盾)

注:

完整的货币及基础单位列表请参见附录。如遇货币未列出，请务必向 PG SOFT 进行确认。

下表说明了基础单位为 1000 的货币之间的关系，以及其对 API、系统、报表和结算流程的影响：

API		获取玩家钱包 转出所有余额	充值 转出	外部数据 抓取工具
实际金额		1000		
游戏显示/ PG SOFT 后台		1		
结算		1000		
运营商 处理 1:1000 换算 (默认)	PG SOFT 发出	1		1
	运营商 发出		1	
PG SOFT 处理 1:1000 换算	PG SOFT 发出	1000		1
	运营商 发出		1000	

9.4 投注类型

投注类型	游戏模式
1	真钱模式

9.5 交易类型

代码	交易类型
1	现金
2	红利游戏
3	免费游戏

9.6 平台

代码	类型	平台
1	网页端	Windows
2		macOS
3		Android
4		iOS
5		其他
6	Cordova	Android
7		iOS
8	Electron	Windows
9		macOS
10	原生应用	Windows
11		macOS
12		Android
13		iOS
98	其他	系统
99		挂机

9.7 语言

代码	语言
en	英文（默认）
en-social	英文（社交语言）
en-stkus	社交娱乐场（适用于 SC 与 GC 货币）
ar-EG	阿拉伯文
az-Latn-AZ	阿塞拜疆文（拉丁字母）
bg-BG	保加利亚文
bn-BD	孟加拉文
cs-CZ	捷克文
da-DK	丹麦文
de-DE	德文
el-GR	希腊文
es-ES	西班牙文
et-EE	爱沙尼亚文
fa-IR	波斯文
fi-FI	芬兰文
fr-FR	法文
hi-IN	印地文
hu-HU	匈牙利文
hy-AM	亚美尼亚文
id-ID	印尼文
it-IT	意大利文
ja-JP	日文
ko-KR	韩文
lo-LA	老挝文
lt-LT	立陶宛文
mn-MN	蒙古文

my-MM	缅甸文
no-NO	挪威文
nl-NL	荷兰文
pl-PL	波兰文
pt-PT	葡萄牙文
pt-BR	巴西葡萄牙文
ro-RO	罗马尼亚文
ru-RU	俄文
sr-Latn-RS	塞尔维亚文 (拉丁字母)
si-LK	僧伽罗文
sk-SK	斯洛伐克文
sq-AL	阿尔巴尼亚文
sv-SE	瑞典文
th-TH	泰文
tr-TR	土耳其文
uk-UA	乌克兰文
ur-PK	乌尔都文
uz-Latn-UZ	乌兹别克文 (拉丁字母)
vi-VN	越南文
zh	中文